MgcCT: Motif-Preserving Graph Clustering on Continuous-Time Temporal Graphs

Longlong Lin
Southwest University
longlonglin@swu.edu.cn

Jin Zhao
Huazhong University of Science and Technology
zjin@hust.edu.cn

Hongchao Qin
Beijing Institute of Technology
qhc.neu@gmail.com

Qiangqiang Dai
Beijing Institute of Technology
qiangd66@gmail.com

Zeli Wang
Chongqing University of Posts and Telecommunications
zlwang@cqupt.edu.cn

ABSTRACT

Temporal graphs naturally model complex real-life dynamic systems, including financial, social, and transportation networks. While temporal graph clustering has achieved significant success, neither continuous topological-temporal dependencies nor higher-order connectivity patterns have been well-explored, leading to sub-optimal performance. To alleviate these dilemmas, we introduce a novel problem called Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*). Namely, given a continuous-time temporal graph and a temporal motif, *MgcCT* aims to detect a high-quality cluster by preserving the connectivity of higher-order structures built up from the temporal motif. To this end, we first propose a new concept of temporal motif conductance, which can be interpreted as the probability of escape from a random walk in the cluster. Then, we formulate *MgcCT* as an optimization task that aims at finding a cluster with minimum temporal motif conductance. Subsequently, we prove that the *MgcCT* problem is NP-hard and propose two efficient algorithms. Specifically, we first develop an approximate global algorithm *AGA* based on a sparse edge-focused weighted graph, which achieves a Cheeger-type approximation guarantee. To further boost efficiency, we then propose a heuristic local algorithm *HLA* that computes edge-focused weights on demand without globally constructing the weighted graph in advance. Finally, extensive empirical results on real-world and synthetic datasets showcase the effectiveness, efficiency, and scalability of our solutions compared to ten competitors.

PVLDB Reference Format:
Longlong Lin, Jin Zhao, Hongchao Qin, Qiangqiang Dai, and Zeli Wang. MgcCT: Motif-Preserving Graph Clustering on Continuous-Time Temporal Graphs. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at XXXXXX.

1 INTRODUCTION

Graph clustering is a fundamental graph data mining task, which has been widely studied in image segmentation [65, 71], load balancing [12], and machine learning [7, 40]. However, real-life graphs often contain complex but useful time information (i.e., the edges are associated with rich timestamps), which provides insights into the underlying mechanisms [27, 28]. For example, in collaboration networks, the timestamps of the edges indicate when the corresponding authors co-authored papers. Therefore, many researchers are working on designing various temporal graph clustering algorithms to simultaneously consider the entanglement of structure and temporal dynamics [11, 55, 58, 66].

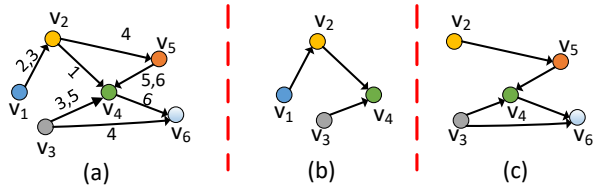


Figure 1: (a) is the continuous-time temporal graph. (b) and (c) are time-aggregated snapshots by aggregating timestamps 1-3 and 4-6.

Despite the significant success of temporal graph clustering, existing methods suffer from two-fold defects. Firstly, they approximately compress the raw continuous-time temporal networks to the discrete time-aggregated snapshots, in which each snapshot consists of edges that occur in the user-specified temporal resolution (e.g., one hour or day). Unfortunately, the discrete and non-continuous temporal graphs seriously affect the effectiveness of approaches [9, 24, 57, 58]. Imagine that when the resolution is too coarse, it may not distinguish paths correctly, which is an important indicator for clustering [16, 36, 53]. Taking Fig. 1 as an example, (v_1, v_2, v_4) of (b) is an invalid path with respect to (w.r.t.) time-order in (a), and the valid path (v_1, v_2, v_5) of (a) cannot be identified by (b) and (c). Besides, it may also break some important temporal structures, such as burst events that occur over a short period of time. Therefore, these existing methods overlook important continuous topological-temporal dependencies. Secondly, they adopt the dyadic temporal edge as the atomic clustering unit to partition the graph, while significantly ignoring the frequently occurring higher-order temporal connectivity patterns (a.k.a., temporal motifs [50, 59]). Actually, unlike temporal edges, each temporal motif is a small subgraph with temporal constraints (e.g., temporal ordering and duration) that indicates some unique functions [33, 49, 74]. For example, in temporal money transfer networks, the cycle structure that follows a chronological order with short duration (named temporal cycle) is an important factor in judging money laundering [34, 37, 67]. The clusters with dense temporal cycles can be used to

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

identify financial criminal gangs [34, 48, 77]. In temporal customer-product networks, the temporal butterfly (i.e., the (2,2)-biclique with temporal constraints) is a significant pattern of buying behavior [8]. The clusters with dense temporal butterflies are suitable to enhance the accuracy of recommendations. Therefore, using such mesoscopic-level temporal motifs as the new clustering unit (i.e., each temporal motif should be regarded as a whole that cannot be split [59]) for temporal graph clustering has significant practical value.

Motif-preserving graph clustering (a.k.a., higher-order graph clustering) can typically be addressed by the following three potential models [42, 44]: motif modularity, motif density, and motif conductance. The first two models only consider the internal higher-order cohesiveness and overlook the important external sparsity, which is an important factor in measuring the clustering quality [25, 31, 36, 41, 51], resulting in unsatisfactory empirical performance (Section 5). This aspect, however, is addressed by the third model [5, 6, 30, 72, 76, 78, 82]. Inspired by this, we move beyond traditional motif conductance by integrating temporal information and formulating a novel concept of temporal motif conductance. Specifically, temporal motif conductance is the ratio of the number of temporal motif instances crossing the cluster boundary to the smaller temporal motif volume of the cluster and its complement (Definition 6). Smaller temporal motif conductance implies the partition preserves rich temporal motif instances inside the cluster and cuts temporal motif instances as little as possible. Besides, temporal motif conductance can be viewed as the escape probability of the cluster (Theorem 2.1), which generalizes the random walk interpretation of conventional conductance [54, 76]. Based on this, we formulate our problem of Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*) as an optimization task that aims at finding a cluster with the minimum temporal motif conductance. Consequently, from the perspectives of both cut and escape probability, *MgcCT* has nice interpretability.

MgcCT is practically important but technically challenging. For example, one simple treatment for solving *MgcCT* is to utilize static higher-order graph clustering methods [6, 30, 72] to temporal networks by ignoring the time information of edges, but such solutions greatly compress the informative time semantics behind temporal graphs and thus obtain sub-optimal clustering qualities (Section 5). Another possible method is to project the original temporal graph into a static weighted graph by enumerating temporal motifs, where each motif induces an edge between every pair of its vertices and contributes a unit weight to each such edge. Then, classic graph clustering algorithms can be applied to partition the resulting weighted graph [6, 42, 78]. However, such weighted graph construction may introduce weighted edges between originally nonadjacent vertices, increasing the cost of weighted-graph construction and subsequent clustering. Besides, it requires an exact enumeration of all temporal motif instances, which incurs prohibitively high time&space overheads (is even NP-hard for the simple temporal star [49]) due to the topological-temporal combinatorial explosions [52, 59, 70, 80]. On the other hand, as delineated in Section 6, many temporal lower-order graph clustering algorithms have been proposed in the literature, encompassing modularity-based [55], conductance-based [66], and density-based [58] methods. However, these approaches overlook the more insightful higher-order substructures, ultimately leading to subpar empirical clustering performance. Thus, existing methods suffer from prohibitive computational costs or sub-optimal qualities.

To overcome these challenges, we first propose an approximate global algorithm *AGA* based on our proposed *edge-focused* weighted graph projection method. Specifically, since the conventional pair-wise motif-based weighted graph may contain weighted edges absent from the original de-temporal graph, resulting in magnification of the original edge information. To prevent this, *AGA* specifies that an edge's weight is non-zero only when it also exists as an edge in the original temporal graph (Definition 9). We further establish motif-dependent cut and volume bounds (Lemmas 4.1 and 4.2) for this sparse representation, yielding a Cheeger-type guarantee for *AGA*(Theorem 4.3). Additionally, unlike existing methods that exhaustively enumerate all temporal motif instances, the edge-focused representation requires only aggregate motif-participation counts on original edges, making it compatible with specialized edge-conditioned counting algorithms. To further boost the efficiency, we devise a heuristic local algorithm, *HLA*, which avoids global edge-focused weighted graph construction in advance. Specifically, *HLA* starts a random walk from a query node to obtain a *local* probability distribution π (i.e., π has very few non-zero elements) by dynamically computing the edge-focused weights on the fly, which greatly reduces the time and space overheads by avoiding unnecessary edge-focused weights in advance. Then, *HLA* executes a *sweep-cut* procedure over π to obtain a heuristic result. In a nutshell, our main contributions are summarized as follows.

- ***A Novel Model.*** We propose a novel problem of Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*) by introducing temporal motif conductance, a higher-order temporal clustering objective that jointly captures internal motif preservation and external motif separability.
- ***Two Efficient Algorithms.*** We prove the NP-hardness of our problem and propose two efficient algorithms. Specifically, we first devise an approximate global algorithm AGA based on a sparse, edge-focused weighted graph projection, for which we establish a Cheeger-style approximation guarantee. Then, a heuristic pure local algorithm *HLA* is proposed, which computes edge-focused weights on demand during local diffusion, striking a better trade-off between efficiency and quality.
- ***Extensive Experiments.*** Empirical results on real-world and synthetic datasets demonstrate our proposed solutions indeed are efficient and effective when contrasted with ten competitors. The source code and datasets of this paper can be found at [3].

2 PRELIMINARIES

2.1 Basic Notations

Let $\mathcal{G}(V, \mathcal{E})$ be a directed continuous-time temporal graph, where V denotes the node set and $\mathcal{E}$ represents the directed temporal edge set. Each temporal edge $(u, v, t) \in \mathcal{E}$ indicates an interaction from node u to node v at a specific timestamp $t \in \mathbb{R}$. Note that (u, v, t_i) and (u, v, t_j) are regarded as two different temporal edges if $t_i \neq t_j$. Namely, nodes u and v can be connected at multiple different timestamps. Let $n = |V|$ and $m = |\mathcal{E}|$ be the number of nodes and temporal edges, respectively. For any node subset $S \subseteq V$, we denote $\mathcal{G}_S(S, \mathcal{E}_S)$ is a temporal subgraph of $\mathcal{G}(V, \mathcal{E})$

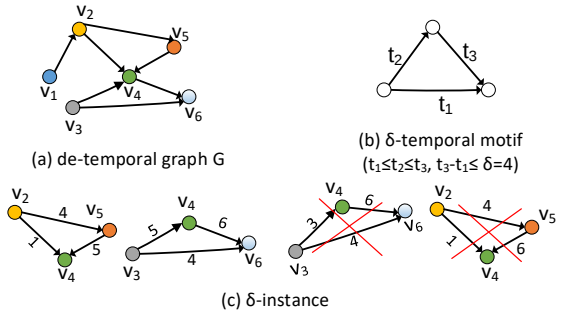


Figure 2: The latter two of (c) are not δ-instances because they respectively violate conditions (i) and (ii) in Definition 2.

iff $\mathcal{E}_S = \{(u,v,t)|u,v \in S\} \subseteq \mathcal{E}$. Compared with previous studies that aggregate temporal edges into discrete snapshots [45–47, 55, 66, 81, 83], the continuous-time temporal graph considered in this paper does not perform any timestamp aggregation on the raw datasets. Thus, in this paper, we examine more general and purely temporal graphs that can preserve fine-grained temporal dynamics in a lossless fashion [9, 24, 57, 58]. For convenience, we use the temporal graph to represent the directed continuous-time temporal graph when the context is clear. We use $G(V,E)$ to denote the de-temporal graph of $\mathcal{G}$, in which $E = \{(u,v)|(u,v,t) \in \mathcal{E}\}$ and $|E| = \bar{m}$. That is, G is a simple static directed graph obtained by discarding timestamps and merging duplicate directed edges of $\mathcal{G}$. Fig. 2(a) shows a de-temporal graph G of Fig. 1(a). Although there are many temporal motif definitions, we use the most representative δ-temporal motif [49, 59, 62, 63, 74] due to its generality, but our results can be easily extended to others [50][1].

Definition 1. ***[δ-temporal motif [59]]*** *A δ-temporal motif $\mathcal{M}_\delta$ is a temporal graph that can be represented as an edge stream $\mathcal{M}_\delta = \{(x_1,y_1,t_1),(x_2,y_2,t_2),...,(x_l,y_l,t_l)\}$ where $t_i \le t_{i+1}$ for all $1 \le i \le l-1$, and $t_l - t_1 \le \delta$.*

In this paper, unless otherwise specified, a δ-temporal motif requires that its de-temporal graph be weakly connected [59].

Definition 2. ***[δ-instance [59]]*** *The temporal subgraph $\mathcal{G}_S = \{(u_1,v_1,t_1'),(u_2,v_2,t_2'),...,(u_l,v_l,t_l')\}$, where $t_i' \le t_{i+1}'$ for all $1 \le i \le l-1$, is called a δ-instance of $\mathcal{M}_\delta$, denoted as $\mathcal{G}_S \in \mathcal{M}_\delta$, if the following conditions are satisfied.*

(i) Temporal ordering: *There exists a bijection h on the nodes such that $h(x_i) = u_i$ and $h(y_i) = v_i$ for $i = 1,2,...,l$;*

(ii) Temporal duration: *The time differences of the temporal edges in $\mathcal{G}_S$ do not exceed δ, i.e., $t_l' - t_1' \le \delta$.*

Fig. 2(b) and (c) show a toy δ-temporal motif and the corresponding δ-instances, respectively.

2.2 Problem Statement

Motif-preserving graph clustering has been extensively studied in static networks, which aims to identify a cluster C such that the vertices within C participate in many motif instances and few motif instances cross C and $\bar{C} = V \setminus C$ [5, 6, 30, 72, 76, 78, 82]. However, existing methods ignore the frequently appearing temporal properties of graphs, resulting in sub-optimal performance (details in Section 5). Inspired by this, we propose a new concept of temporal motif conductance, generalizing the motif conductance by integrating temporal information.

Definition 3. ***[temporal motif cut]*** *Given a node subset C and a δ-temporal motif $\mathcal{M}_\delta$, the temporal motif cut of C is denoted by $cut_{\mathcal{M}_\delta}(C) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(S \cap C \neq \emptyset \& S \cap \bar{C} \neq \emptyset)$. Where $I(.)$ is the indicator function.*

Definition 4. ***[temporal motif degree]*** *Given a δ-temporal motif $\mathcal{M}_\delta$ and a node u, the temporal motif degree of node u is denoted by $d_{\mathcal{M}_\delta}(u) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(u \in S)$.*

Equivalently, $cut_{\mathcal{M}_\delta}(C)$ is the cardinality of $\{\mathcal{G}_S \in \mathcal{M}_\delta : S \cap C \neq \emptyset,\ S \cap \bar{C} \neq \emptyset\}$, while $d_{\mathcal{M}_\delta}(u)$ is the cardinality of $\{\mathcal{G}_S \in \mathcal{M}_\delta : u \in S\}$. We retain the indicator notation for convenience in subsequent proofs.

Definition 5. ***[temporal motif volume]*** *Given a node subset C and a δ-temporal motif $\mathcal{M}_\delta$, the temporal motif volume of C is denoted by $vol_{\mathcal{M}_\delta}(C) = \sum_{u \in C} d_{\mathcal{M}_\delta}(u)$.*

Definition 6. ***[temporal motif conductance]*** *Given a node set C and a δ-temporal motif $\mathcal{M}_\delta$, the temporal motif conductance of C is denoted by $\phi_{\mathcal{M}_\delta}(C) = \frac{cut_{\mathcal{M}_\delta}(C)}{\min\{vol_{\mathcal{M}_\delta}(C), vol_{\mathcal{M}_\delta}(\bar{C})\}}$. For completeness, if the denominator is zero, we set $\phi_{\mathcal{M}_\delta}(C) = +\infty$.*

Example 1. *Consider the two valid δ-instances in Fig. 2(c) and let $C = \{v_2, v_4, v_5\}$. One instance is entirely contained in C, while the other crosses its boundary. Thus, $cut_{\mathcal{M}_\delta}(C) = 1$ according to Definition 3. Besides, by Definition 4, we can know that the temporal motif degrees of v_2, v_4, and v_5 are 1, 2, and 1, respectively. Hence, $vol_{\mathcal{M}_\delta}(C) = 4$ by Definition 5. Since $vol_{\mathcal{M}_\delta}(\bar{C}) = 2$, we obtain $\phi_{\mathcal{M}_\delta}(C) = \frac{1}{\min\{4,2\}} = 0.5$ by Definition 6.*

Problem formulation. Given a temporal graph $\mathcal{G}(V,\mathcal{E})$ and a δ-temporal motif $\mathcal{M}_\delta$, the goal of the Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*) is to identify a vertex subset C^* such that $\phi_{\mathcal{M}_\delta}(C^*) \le \phi_{\mathcal{M}_\delta}(C)$ for any $C \subseteq V$. We use $\phi^*_{\mathcal{M}_\delta}$ to represent $\phi_{\mathcal{M}_\delta}(C^*)$.

Our full paper [3] provides more advantages of *MgcCT*.

2.3 Problem Analysis

This subsection enhances our problem through the lens of escape probability. Then, we prove the problem's NP-hardness.

Definition 7. ***[cluster-constrained temporal motif walk]*** *Given a temporal graph $\mathcal{G}(V,\mathcal{E})$, a δ-temporal motif $\mathcal{M}_\delta$ with k nodes, and a cluster $C \subseteq V$, we define a one-step cluster-constrained temporal motif walk as follows. Given the current node u: (1) If $d_{\mathcal{M}_\delta}(u) = 0$, the walker stays at u. (2) Otherwise, the walker first samples a δ-instance $\mathcal{G}_S \in \mathcal{M}_\delta$ containing u with probability $\Pr(\mathcal{G}_S \mid u) = \frac{I(u \in S)}{d_{\mathcal{M}_\delta}(u)}$. (3) If $S \subseteq C$ or $S \subseteq \bar{C}$, the walker stays at u. (4) Otherwise, let $i = |S \cap C|$ with $1 \le i \le k-1$. If $u \in C$, then with probability $\frac{1}{i}$, the walker moves to a node chosen uniformly at*

[1] Since our models and algorithms only depend on temporal motif instances, other temporal motif definitions can be directly extended to our solutions by the corresponding enumeration or counting algorithms.

random from $S \cap \bar{C}$; with the remaining probability $1 - \frac{1}{i}$, it stays at u. If $u \in \bar{C}$, then with probability $\frac{1}{k-i}$, the walker moves to a node chosen uniformly at random from $S \cap C$; with the remaining probability $1 - \frac{1}{k-i}$, it stays at u.

Remark. The factors $\frac{1}{i}$ and $\frac{1}{k-i}$ are crucial. For a crossing δ-instance $\mathcal{G}_S$ with $|S \cap C| = i$, the same instance is counted once for each of its i nodes in C and once for each of its $k - i$ nodes in $\bar{C}$. Dividing by these multiplicities ensures that each crossing δ-instance contributes exactly once to the total escape probability. Therefore, the following theorem gives an *exact* probabilistic interpretation of temporal motif conductance.

The detailed proofs of the following lemmas and theorems are put in the full paper [3] due to space limitations.

THEOREM 2.1. ***[escape probability]*** *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ-temporal motif $\mathcal{M}_\delta$ with k nodes, and a cluster C. Consider the cluster-constrained temporal motif walk in Definition 7. If the initial state w_0 is sampled from C (or $\bar{C}$) according to the temporal motif degree distribution, we have $\phi_{\mathcal{M}_\delta}(C) = \max\left\{\Pr(w_1 \in \bar{C} \mid w_0 \in C), \Pr(w_1 \in C \mid w_0 \in \bar{C})\right\}$.*

By Theorem 2.1, temporal motif conductance admits an exact one-step escape-probability interpretation. Concretely, for a cluster C, its temporal motif conductance equals the larger of the two probabilities that the above walk moves from C to $\bar{C}$ or from $\bar{C}$ to C in one step.

THEOREM 2.2. ***[Hardness]*** *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ-temporal motif $\mathcal{M}_\delta$, the problem of identifying MgcCT from $\mathcal{G}(V, \mathcal{E})$ is NP-hard.*

PROOF SKETCH. We reduce the NP-hard minimum static motif conductance problem [6] to *MgcCT*. Given a static graph G and a motif M, we assign the same timestamp to every edge and construct the corresponding temporal motif with $\delta = 0$. Since equal timestamps are permitted, static motif instances correspond to valid temporal motif instances under the same matching convention. Thus, the motif cut and volume, and consequently motif conductance, are preserved for every vertex subset C. The reduction is polynomial, establishing NP-hardness of general *MgcCT*. This hardness arises from conductance minimization, not motif-instance matching. □

3 EXISTING SOLUTIONS

According to Theorem 2.2, the intrinsic complexity of the proposed problem necessitates the development of approximate algorithms to efficiently optimize our objectives. At a high level, most existing methods adhere to the well-known two-stage clustering framework [6, 42, 72, 78]. As shown in Figure 3, in the first stage, the input graph is transformed into a static weighted graph, where the weight of each edge corresponds to the number of motif instances in which the edge participates. In the second stage, the static weighted graph is partitioned using any graph clustering algorithm. Different algorithms differentiate themselves in how they efficiently and effectively partition the static weighted graph. For instance, Benson et al. [6] and Yin et al. [78] employ the Fiedler vector-based global spectral clustering algorithm and the local diffusion graph clustering algorithm, respectively, to partition the static weighted graph. Building on this idea, we introduce

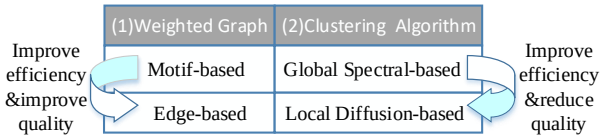


Figure 3: Key functions of the seminal two-stage framework.

the following key concepts. Let $\mathcal{A} \in \mathbb{R}^{n \times n}$ denote the standard zero-diagonal δ-temporal motif weighted graph (matrix), where $\mathcal{A}_{uv} = \mathcal{A}_{vu} = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(u \in S \wedge v \in S)$ for $u \neq v$, and $\mathcal{A}_{uu} = 0$. *It is crucial to note that u and v may not necessarily be connected by an edge in the original de-temporal graph (i.e., $(u, v) \notin E$).* Using these notations, several basic existing solutions are outlined below (details in their original papers).

Cheeger Guarantee for the Standard Projection. For a k-vertex motif instance with i vertices in C and $k - i$ in $\bar{C}$, the instance contributes $i(k-i)$ to the cut of $\mathcal{A}$. Since $k-1 \leq i(k-i) \leq \lfloor k^2/4 \rfloor$, we obtain $(k-1)cut_{\mathcal{M}_\delta}(C) \leq cut_{\mathcal{A}}(C) \leq \lfloor k^2/4 \rfloor cut_{\mathcal{M}_\delta}(C)$. Moreover, $vol_{\mathcal{A}}(C) = (k-1)vol_{\mathcal{M}_\delta}(C)$. Consequently, $\phi_{\mathcal{M}_\delta}(C) \leq \phi_{\mathcal{A}}(C) \leq \frac{\lfloor k^2/4 \rfloor}{k-1}\phi_{\mathcal{M}_\delta}(C)$. The classical Cheeger inequality therefore gives an $O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$ guarantee for any fixed motif size. The complete derivation is provided in the full version [3].

3.1 Order-Based Solutions

These solutions can be summarized in two steps: (1) Sort all vertices according to a predefined rule. Without loss of generality, let the sorted vertices be denoted as $u_1, u_2, \ldots, u_n$. (2) Output $S = \arg\min \phi_{\mathcal{A}}(S_i)$, where $S_i = \{u_1, u_2, \ldots, u_i\}$ and $\phi_{\mathcal{A}}(S_i)$ represents the traditional edge-based conductance [43, 73] of S_i on the weighted graph $\mathcal{A}$. Specifically, $\phi_{\mathcal{A}}(S_i) = \frac{\sum_{u \in S_i, v \notin S_i} \mathcal{A}_{uv}}{\min\{\sum_{u \in S_i} \mathcal{D}_{uu}, \sum_{v \notin S_i} \mathcal{D}_{vv}\}}$, where $\mathcal{D}_{uu} = \sum_v \mathcal{A}_{uv}$. Consequently, the following two order-based solutions are proposed: *THSC* [6] and *THLC* [78]. In particular, *THSC* [6] and *THLC* [78] utilize the Fiedler Vector and Personalized PageRank vector, respectively, to sort all vertices. In our continuous-time setting, THSC and THLC adapt existing motif-based spectral clustering and local diffusion frameworks, respectively. Their weighted graph $\mathcal{A}$ is constructed from valid δ-temporal motif instances, while the original spectral and Personalized PageRank procedures are retained. Similarly, the following THPR (Section 3.2) uses the corresponding temporal-motif statistics in its peeling procedure. Table 1 summarizes these basic solutions.

Limitations of Order-Based Solutions. From Table 1, we observe the following: (1) Although *THSC* admits a Cheeger-type guarantee for any fixed motif size, its conventional pairwise motif-weighted graph may contain weighted edges absent from the original de-temporal graph, leading to additional construction and clustering overhead. (2) *THLC* is a heuristic algorithm, and its clustering quality is significantly influenced by numerous hard-to-tune parameters and seeding strategies. In practical applications, its performance can be unstable and may even yield very poor solutions, as demonstrated by our experimental results. (3) *THSC* and *THLC* suffer from significant computational and memory inefficiencies due to their need to enumerate and store all temporal motif instances.

Table 1: A comparison of *MgcCT* algorithms. × indicates no approximation guarantee. $\phi^*_{\mathcal{M}_\delta} \in (0,1)$ is the minimum temporal motif conductance. $E(\mathcal{M}_\delta)$ denotes the cost of enumerating temporal motif instances and accumulating their contributions to motif-based weights, whereas $C(\mathcal{M}_\delta)$ denotes the cost of computing all exact edge-focused weights using an exact weight-computation routine. $S_A = nnz(\mathcal{A})$ is the number of nonzeros in the pairwise weighted matrix. $T^{(2)}_{\text{eig}}(X)$ denotes the cost of computing the required second eigenvector. T_{PPR} and T_{peel} denote the costs of PPR-based clustering and motif-based peeling, respectively. $\Delta_{\max}$, α, and ϵ follow Theorem 4.4.

Methods	Accuracy Guarantee	Time Complexity	Space Complexity	Weighted Graph
THSC [6]	$O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$	$O(E(\mathcal{M}_\delta) + T^{(2)}_{\text{eig}}(\mathcal{A}) + S_A + n\log n)$	$O(S_A)$	Motif-based
THLC [78]	×	$O(E(\mathcal{M}_\delta) + T_{\text{PPR}})$	$O(S_A)$	Motif-based
THPR [42]	$O(\frac{1}{2} + \frac{1}{2}\phi^*_{\mathcal{M}_\delta})$	$O(E(\mathcal{M}_\delta) + T_{\text{peel}})$	$O(m)$	Motif-based
AGA (Our)	$O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$	$O(C(\mathcal{M}_\delta) + T^{(2)}_{\text{eig}}(W) + \bar{m} + n\log n)$	$O(m)$	Edge-based
HLA (Our)	×	$O(\frac{\Delta_{\max}}{\alpha\epsilon})$	$O(\frac{1}{\alpha\epsilon})$	Edge-based

3.2 The Peeling-based Solution

Recently, Lin et al. [42] presented a peeling-based method that seamlessly integrates the computation and partitioning of $\mathcal{A}$ in an iterative fashion, yielding results that come with approximation guarantees. Specifically, they redefine the concept of motif conductance, where temporal motif conductance represents a specialized instance when the motif is a δ-temporal motif, and introduce a peeling-based greedy algorithm that iteratively refines motif conductance by initiating the optimization process from each individual vertex. To align with the requirements of our problem, we provide the following definitions.

Definition 8. *(Temporal Motif Resident) For a vertex set C, the temporal motif resident of a vertex $u \in C$ with respect to C is defined as:* $Tr_C(u) = \frac{d_{\mathcal{M}_\delta}(u) + d_{\mathcal{M}_\delta}(u,C) - d_{\mathcal{M}_\delta}(u,\bar{C})}{d_{\mathcal{M}_\delta}(u)}$, *in which* $d_{\mathcal{M}_\delta}(u,H) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(u \in S, S - \{u\} \subseteq H)$ *and H can be either C or $\bar{C}$.*

By Definition 8, the Temporal Higher-order Peeling Removal (*THPR*) algorithm is outlined as follows: First, it calculates the temporal motif resident for each vertex. Subsequently, it iteratively eliminates the vertex with the smallest temporal motif. Finally, it outputs the result that exhibits the smallest temporal motif conductance during the iteration procedure.

Limitations of the Peeling-Based Approach. As indicated in Table 1, although *THPR* offers results with approximation guarantees, our empirical evaluation reveals that it exhibits suboptimal practical clustering quality. More importantly, *THPR* imposes excessively high time costs due to the necessity of enumerating all temporal motif instances, which leads to a combinatorial explosion and diminished efficiency.

4 THE PROPOSED SOLUTIONS FOR *MGCCT*

This section first constructs a novel weighted graph to approximate temporal motif cut and temporal motif volume, and then proposes a global algorithm with theoretical approximation guarantees. Finally, we present a heuristic local algorithm for scalability.

4.1 Novel Edge-focused Weighted Graph

As discussed in Section 3, the conventional pairwise motif-weighted graph also admits a Cheeger-type guarantee for any fixed motif size. Nevertheless, it may introduce weighted connections between vertices that are not adjacent in the original de-temporal graph, resulting in a potentially dense representation. Motivated by this observation, we investigate whether temporal motif cut and volume can be approximated using a sparse weighted graph supported only on original de-temporal edges. To this end, we propose the following edge-focused weighted graph projection.

Before introducing the formal definition, we briefly highlight the motivation behind our edge-focused representation. Existing higher-order clustering methods [6, 42, 78] may construct motif-based weighted graphs or maintain motif-related statistics by processing temporal-motif instances. In contrast, our edge-focused projection stores aggregate motif-participation counts only on original de-temporal edges. As established in Lemmas 4.1 and 4.2, the resulting edge-focused weighted graph preserves temporal motif cut and volume within motif-dependent factors. This representation avoids storing the complete collection of motif instances and supports sparse weighted-graph construction.

Definition 9. ***[edge-focused weighted matrix]*** *Given a temporal graph $\mathcal{G}$ with its de-temporal graph $G = (V, E)$ and a δ-temporal motif $\mathcal{M}_\delta$, the edge-focused weighted matrix $W_{ef} \in R^{n\times n}$ is defined:*

$$(W_{ef})_{uv} = \begin{cases} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I((u,v) \in E^{dt}_S), & (u,v) \in E \\ 0, & (u,v) \notin E \end{cases} \quad (1)$$

Where E^{dt}_S denotes the directed de-temporal edge set of a temporal motif instance $\mathcal{G}_S \in \mathcal{M}_\delta$.

Example 2. *Consider the four-node temporal path motif M_3 in Fig. 4 with $\delta = 4$ and the temporal graph in Fig. 1(a). There are six valid temporal motif instances in this example. The conventional pairwise projection produces nine distinct weighted vertex pairs, including (v_1, v_4), whose endpoints are not directly connected in the original de-temporal graph. In contrast, the edge-focused projection assigns nonzero weights to only four original directed edges: (v_1, v_2), (v_2, v_5), (v_5, v_4), and (v_4, v_6). Thus, the proposed representation preserves motif-participation information on the original edge support, without introducing additional motif-induced connections. The complete instance enumeration and weight calculations are provided in the full version [3].*

Remark. By Definition 9, it is apparent that the distinctive feature of our edge-focused weighted matrix (graph) in comparison to existing motif-based weighted graphs is that it exclusively assigns weights to edges that are present in the original graph. Specifically, if an edge (u, v) does not exist in E, its edge-focused weight is set to zero. In essence, an edge in the edge-focused weighted graph (matrix) possesses a non-zero weight if and only if it also appears as an edge in the original de-temporal graph G and participates in at least one temporal motif instance. We underscore that, despite its simplicity, this modification engenders several substantial benefits: (1) The resulting sparse representation admits motif-dependent cut and volume bounds and a corresponding Cheeger-type approximation guarantee. (2) In the worst-case scenario, the size of the edge-focused weighted graph W_{ef} is $O(\bar{m})$, which is more compact than the motif-based weighted graph $\mathcal{A}$, whose size is $O(\min\{n^2, \binom{k}{2} num(\mathcal{M}_\delta)\})$, where $num(\mathcal{M}_\delta)$ represents the number of motif instances. Thus, the edge-focused representation can reduce the storage and computational costs of subsequent clustering, particularly when the conventional pairwise motif-based projection introduces substantially more nonzero entries.

Let $H_\delta = (V_\delta, E_\delta)$ be the directed de-temporal graph of $\mathcal{M}_\delta$, where $|E_\delta| = m_\delta$. For any nonempty proper subset $X \subset V_\delta$, define $\partial_{H_\delta}(X) = \{(a, b) \in E_\delta \mid a \in X, b \notin X \text{ or } a \notin X, b \in X\}$. That is, $\partial_{H_\delta}(X)$ denotes the set of directed de-temporal motif edges crossing the cut $(X, V_\delta \setminus X)$, regardless of edge directions. We define $\underline{c}_\delta = \min_{\emptyset \neq X \subsetneq V_\delta} |\partial_{H_\delta}(X)|, \overline{c}_\delta = \max_{\emptyset \neq X \subsetneq V_\delta} |\partial_{H_\delta}(X)|$. Similarly, for each motif vertex $x \in V_\delta$, define its incident de-temporal degree as $d_{H_\delta}(x) = |\{(x, y) \in E_\delta\}| + |\{(y, x) \in E_\delta\}|$. Let $\underline{d}_\delta = \min_{x \in V_\delta} d_{H_\delta}(x), \overline{d}_\delta = \max_{x \in V_\delta} d_{H_\delta}(x)$. The following pivotal lemmas that illuminate the correlation between $cut_{\mathbb{M}_\delta}(.)$ and $vol_{\mathbb{M}_\delta}(.)$ within the original temporal graph $\mathcal{G}$ and our proposed edge-focused weighted graph W_{ef}, which establish a solid theoretical foundation that underpins the development of our subsequent algorithms.

LEMMA 4.1. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a cluster $C \subseteq V$, and a δ-temporal motif $\mathcal{M}_\delta$. Let $W = W_{ef} + W_{ef}^\top$, we have*

$$\underline{c}_\delta \operatorname{cut}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C,\, v \notin C} W_{uv} \leq \overline{c}_\delta \operatorname{cut}_{\mathcal{M}_\delta}(C).$$

LEMMA 4.2. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a cluster $C \subseteq V$, and a δ-temporal motif $\mathcal{M}_\delta$. Let $W = W_{ef} + W_{ef}^\top$, we have*

$$\underline{d}_\delta \operatorname{vol}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C} \sum_{v \in V} W_{uv} \leq \overline{d}_\delta \operatorname{vol}_{\mathcal{M}_\delta}(C).$$

Implications of Lemma 4.1 and Lemma 4.2. Lemma 4.1 and Lemma 4.2 show that the weighted graph $W = W_{ef} + W_{ef}^\top$ preserves both the temporal motif cut and the temporal motif volume up to motif-dependent constants. Specifically, the cut approximation is controlled by the motif cut-crossing constants $\underline{c}_\delta$ and $\overline{c}_\delta$, while the volume approximation is controlled by the motif incident-degree constants $\underline{d}_\delta$ and $\overline{d}_\delta$. As a result, when spectral clustering is applied to W, it offers provable approximation guarantees(Theorem 4.3).

Algorithm 1 Approximate Global Algorithm (*AGA*)

Input: A temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ-temporal motif $\mathcal{M}_\delta$
Output: A cluster C

1: $G \leftarrow$ the de-temporal graph of $\mathcal{G}$
2: Initialize sparse W_{ef} and the diagonal degree vector $\mathcal{D}$
3: **for** each edge $(u, v) \in G$ **do**
4: $\quad (W_{ef})_{uv} \leftarrow$ the edge-focused weight by Equation 1
5: $W \leftarrow W_{ef} + W_{ef}^\top$
6: **for** each $u \in V$ **do**
7: $\quad \mathcal{D}_{uu} \leftarrow \sum_{v \in V} W_{uv}$
8: $\mathcal{L} \leftarrow I - \mathcal{D}^{-1/2} W \mathcal{D}^{-1/2}$
9: $z \leftarrow$ eigenvector of the second smallest eigenvalue of $\mathcal{L}$
10: $z_i \leftarrow$ the index of z with ith smallest value
11: $C \leftarrow \arg\min \phi_W(C_i)$, where $C_i = \{z_1, z_2, \ldots, z_i\}$ and $\phi_W(C_i)$ is the edge-based conductance of C_i in W
12: **return** C

4.2 *AGA*: The Approximate Global Algorithm

Based on the preceding analyses, we propose an approximate global algorithm, denoted as *AGA*, which is detailed in Algorithm 1. Specifically, Algorithm 1 initiates by computing the edge-focused weighted matrix W_{ef} (Lines 1-4). Subsequently, we construct a symmetric weighted matrix W by $W \leftarrow W_{ef} + W_{ef}^\top$ (Line 5) and derive the diagonal degree matrix $\mathcal{D}$ with $\mathcal{D}_{uu} = \sum_{v \in V} W_{uv}$ (Lines 6-7). Algorithm 1 then constructs the symmetric normalized Laplacian $L = I - D^{-1/2} W D^{-1/2}$ (Line 8). For spectral partitioning (Lines 9-11), we adopt the direct eigenvector-ordering procedure described in the Fiedler-vector-based spectral clustering subsection of [43] and Section 2 of [72]. Specifically, we compute an eigenvector z associated with the second smallest eigenvalue of $\mathcal{L}$, and obtain a vertex ordering satisfying $z_1 \leq z_2 \leq \cdots \leq z_n$. We then evaluate the prefix sets $C_i = \{z_1, z_2, \ldots, z_i\}$ and select $C = \arg\min_{1 \leq i < n} \phi_W(C_i)$, where $\phi_W(C_i) = \frac{\sum_{u \in C_i, v \notin C_i} (W)_{uv}}{\min\{\sum_{u \in C_i} \sum_{v \in V} (W)_{uv}, \sum_{u \notin C_i} \sum_{v \in V} (W)_{uv}\}}$. Finally, Algorithm 1 outputs C as the target cluster (Line 12).

Remark. The symmetrization step in Line 5 is introduced only to enable the use of the standard normalized Laplacian and the classical Cheeger inequality [1] (see Theorem 4.2) for undirected weighted graphs. It does *not* convert the original temporal graph or the query temporal motif into an undirected one. More specifically, all temporal motif instances are still matched on the original directed continuous-time temporal graph according to Definitions 1 and 2, where both temporal ordering and edge directions are preserved. The directed edge-focused weight $(W_{ef})_{uv}$ is computed before symmetrization and counts only directed temporal motif instances in which the directed edge (u, v) participates. The symmetrized matrix $W = W_{ef} + W_{ef}^\top$ should therefore be understood as a final clustering projection rather than a redefinition of the temporal motif semantics. It aggregates the two directed edge-focused contributions between a pair of vertices only for spectral partitioning. This design provides a clean Cheeger-style analysis while preserving directional and temporal constraints during the weight construction stage.

THEOREM 4.1. *The time (resp., space) complexity of Algorithm 1 is $O(C(\mathcal{M}_\delta) + T_{\text{eig}}^{(2)}(W) + \bar{m} + n \log n)$ (resp., $O(m)$). $C(\mathcal{M}_\delta)$ is the time of computing all exact edge-focused weights using an exact weight-computation routine and $T_{\text{eig}}^{(2)}(W)$ is the time of obtaining the second smallest eigenvector of $\mathcal{L}$ in Line 9 of Algorithm 1.*

Remark. We use a sparse iterative eigensolver based on the Lanczos method [22] to compute the required eigenvector. Each sparse matrix–vector multiplication costs $O(n+\bar{m})$ time because $\mathrm{nnz}(W) = O(\bar{m})$. If the solver converges in t_L iterations, the matrix–vector multiplication cost is $O(t_L(n+\bar{m}))$. Besides, by Table 1, it is clear that existing methods depend on temporal motif enumeration algorithms to obtain the hyperedges (i.e., temporal motif instances) and then construct the motif-based weighted graph for clustering. In contrast, *AGA* directly accumulates motif participation on original de-temporal edges without storing the complete collection of motif instances, which can significantly reduce the computational costs. Furthermore, as stated in Theorem 4.1, *AGA* necessitates only linear space complexity for the storage of basic data structures. This is unlike other algorithms (except for *THPR* [42]), which typically need to store all temporal motif instances, resulting in high space complexity. Consequently, our proposed *AGA* algorithm surpasses SOTA methods in terms of efficiency, as stated in our experiments.

THEOREM 4.2. *Given a temporal graph $\mathcal{G}(V,\mathcal{E})$ and a δ-temporal motif $\mathcal{M}_\delta$. Let λ_2 be the second smallest eigenvalue of $\mathcal{L}$ (Line 8), we have $\frac{d_\delta}{2\bar{c}_\delta}\lambda_2 \le \phi^*_{\mathcal{M}_\delta} \le \frac{\bar{d}_\delta}{\underline{c}_\delta}\sqrt{2\lambda_2}$.*

THEOREM 4.3. *Given a temporal graph $\mathcal{G}(V,\mathcal{E})$ and a δ-temporal motif $\mathcal{M}_\delta$. Let C be the cluster returned by Algorithm 1, we have $\phi_{\mathcal{M}_\delta}(C) \le 2\frac{\bar{d}_\delta}{\underline{c}_\delta}\sqrt{\frac{\bar{c}_\delta}{\underline{d}_\delta}\phi^*_{\mathcal{M}_\delta}}$. In particular, when the temporal motif is fixed, Algorithm 1 achieves a Cheeger-style quadratic approximation guarantee $\phi_{\mathcal{M}_\delta}(C) = O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$.*

PROOF SKETCH. Algorithm 1 uses the direct eigenvector ordering described in Section 4.2. The graph-level result invoked here is the sweep bound $\phi_W(C) \le \sqrt{2\lambda_2}$, as stated in Theorem 2 of [43] and the discussion following Theorem 1 in Section 2 of [72]. Combining this cited graph-level bound with Lemmas 4.1 and 4.2, we obtain $\phi_{\mathcal{M}_\delta}(C) \le \frac{\bar{d}_\delta}{\underline{c}_\delta}\sqrt{2\lambda_2}$. Moreover, by Cheeger's inequality [1], we have $\lambda_2 \le 2\phi^*_W \le 2\frac{\bar{c}_\delta}{\underline{d}_\delta}\phi^*_{\mathcal{M}_\delta}$. Consequently, we have $\phi_{\mathcal{M}_\delta}(C) \le 2\frac{\bar{d}_\delta}{\underline{c}_\delta}\sqrt{\frac{\bar{c}_\delta}{\underline{d}_\delta}\phi^*_{\mathcal{M}_\delta}}$ and $\phi_{\mathcal{M}_\delta}(C) = O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$. □

Remark. By Table 1, our proposed algorithm *AGA* attains an asymptotically equivalent level of clustering quality when compared to the *THSC* method. Both *THSC* and *AGA* admit Cheeger-type guarantees for fixed motifs, but they rely on different weighted representations. The main distinction of *AGA* lies in the sparse support of its edge-focused weighted graph. In contrast, evaluating the clustering quality of *AGA* against *THPR* presents a more intricate analysis, as the performance is contingent upon $\phi^*_{\mathcal{M}_\delta}$. Nevertheless, as outlined in Section 5 , AGA achieves more favorable TMC values than THPR, a phenomenon that can be attributed to the fact that *THPR* is designed with a generic graph focus, thereby neglecting the unique temporal dynamics inherent in temporal graphs.

4.3 *HLA*: The Heuristic Local Algorithm

Although our global approximate algorithm *AGA* is substantially faster than existing enumeration-based methods, it still needs to precompute $(W_{ef})_{uv}$ for every edge (u,v) in the de-temporal graph, and hence must access the entire graph before returning the target cluster. On massive temporal graphs, such a global procedure can still be expensive for interactive applications. For example, on the SO dataset with nearly 50 million temporal edges, *AGA* still requires hours to process one query (Section 5).

Fortunately, on most real-world graphs, the target clusters are often localized, meaning that the proportion of the vertices belonging to the target cluster is very small [6, 42, 44, 78]. As an empirical observation (shown in Table 2), the clusters returned by *AGA* under several representative motifs are small relative to the entire graph. In particular, the ratio of the number of vertices in the target cluster comprises no more than 5% of the total vertices in the original graph. Besides, in numerous real-world scenarios, users are often willing to tolerate a certain degree of inaccuracy in exchange for faster response times, especially in large networks. Therefore, beyond improving the asymptotic complexity of global clustering, it is also desirable to design a *pure local* algorithm whose cost depends mainly on the explored region rather than the entire graph, striking a better balance between efficiency and quality.

A natural idea is to adopt the well-known local graph diffusion framework. However, existing local higher-order clustering methods are not purely local in our setting. For instance, [78] still requires globally enumerating temporal motif instances and materializing a weighted graph as an index before the graph diffusion phase. Consequently, its query processing is local only after paying a substantial global preprocessing cost. In contrast, we seek a method that performs *no global motif enumeration, no weighted-graph materialization, and no external indexing*. To this end, we propose a heuristic local algorithm, denoted by *HLA*, that computes edge-focused weights *on demand* during the diffusion process. The key novelty of *HLA* is not merely replacing the weight definition in a standard local diffusion routine. Instead, *HLA* tightly couples local propagation with on-demand evaluation of the edge-focused weighted graph. Specifically, when a vertex becomes active during the diffusion process, *HLA* computes its edge-focused weighted degree and the edge-focused weights of its incident edges only at that moment. These locally computed quantities are then reused, avoiding repeated computation while never requiring the full weighted graph to be constructed. As a result, *HLA* only touches a small fraction of the graph around the query node, which makes it a genuinely *pure local*, *index-free*, and *on-demand* algorithm.

Before introducing *HLA*, we briefly review the well-known *Push* subroutine [2], which is used to approximate the Personalized PageRank vector. The *Push* algorithm initiates its operation from a source state (node) s and proceeds to disseminate information locally. Throughout its execution, the algorithm iteratively refines two key variables associated with each state u: its reserve $\pi(u)$ and its residue $r(u)$. Specifically, $\pi(u)$ serves as an approximation of the Personalized PageRank value of state u with respect to the source state s, while $r(u)$ quantifies the information that remains to be propagated from state u to other states. In each iteration (Lines 19-24 of Algorithm 2), the *Push* algorithm identifies states u that require information dissemination (Line 20). For each such state u, the algorithm transfers a portion $\alpha r(u)$ to the reserve $\pi(u)$ (Line 24), while the remaining $(1-\alpha)r(u)$ is distributed among the neighboring states in proportion to the edge weights connecting them, rather than being evenly distributed (Lines 21-22). After finishing the

Table 2: The ratio of the number of vertices in the identified cluster to that of the whole graph.

Dataset	# vertices	M_1	M_2	M_3
CM	1,893	46(2.43%)	15(0.79%)	95(5.01%)
UB	152,599	12(0.0078%)	55(0.036%)	14(0.0091%)
SU	189,191	108(0.057%)	403(0.21%)	6(0.0032%)
SO	2,572,345	41(0.0015%)	14(0.00054%)	49(0.0019%)

propagation, the algorithm resets the residue $r(u)$ to zero (Line 24). Following [2, 78], ϵ is used to balance quality and efficiency.

Based on these backgrounds, *HLA* is described in Algorithm 2. In Line 1, it initializes the residue $r(s) = 1$ for the source node s and implicitly sets $r(u) = 0$ for all $u \neq s$, while the reserve $\pi(u)$ is initialized as zero for all $u \in V$. In implementation, r and π are maintained as sparse vectors, where unstored entries are implicitly treated as zero. This avoids any global initialization cost and preserves the locality of *HLA*. In Line 2, it initializes the queue $Q = \{s\}$ as the nodes to be pushed, and the set $D = \emptyset$ as the nodes whose weight information has already been calculated to avoid recomputing the edge-focused weighted degree of a vertex. In Lines 3-8, it locally calculates edge-focused weighted degree and edge weights on demand to perform the push operation (Line 8, Lines 19-24) until the queue Q becomes empty (Line 3). In particular, Line 4 pops a vertex u from the queue Q and then determines whether the edge-focused weighted degree of u has been calculated. If not (Line 5), it adds u into set D (Line 6) and executes Line 7 to obtain the edge-focused weighted degree of u and the edge-focused weight of its neighbor (Lines 14-18). Importantly, this computation is performed *only when u is first activated*, rather than for all nodes in advance. After that, *HLA* executes a push step on u (Line 8, Lines 19-24). Once the propagation terminates, it updates the reserve $\pi(u)$ for all $u \in D$ through edge-focused weighted degree normalization (Lines 9-10). Finally, similar to Lines 10-11 of Algorithm 1, Algorithm 2 also performs the sweep-cut operation to obtain the target cluster C (Lines 11-12). Note that Line 11 only sorts all non-zero entries in π such that $\pi_1 \geq \pi_2 \geq ... \geq \pi_{sup(\pi)}$ ($sup(\pi)$ is the number of the non-zero entries in π and $sup(\pi) << n$ [2]), which guarantees the locality of Algorithm 2.

THEOREM 4.4. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ-temporal motif $\mathcal{M}_\delta$, and two parameters α and ϵ, the time complexity and space complexity of Algorithm 2 are $O(\frac{\Delta_{max}}{\alpha\epsilon})$ and $O(\frac{1}{\alpha\epsilon})$, respectively. Where $\Delta(u)$ is the time cost of calculating the edge-focused weighted degree of node u and $\Delta_{max} = \max\{\Delta(u)|u \in V\}$.*

Remark. While the proposed *HLA* lacks a formal guarantee of accuracy (though its practical performance is comparable to the baselines in our empirical results) because it is based on the heuristic local clustering framework [2, 43, 78], its main advantage is being a strictly pure local, index-free, and on-demand algorithm. It is worth noting that although the theoretical worst-case time complexity bound in Theorem 4.4 involves a global parameter Δ_{max}, this is strictly for analytical purposes to upper-bound the cost. In practice, *HLA* never precomputes or scans the entire graph to obtain Δ_{max}. Besides, unlike existing local higher-order methods that rely on a globally precomputed motif index or weighted graph, *HLA* answers a query without any global preprocessing. As a result, the actual runtime of *HLA* is mainly determined by the locally explored region and the associated on-demand weight computations. Nevertheless, with the default $\epsilon = 1/m$, Theorem 4.4 does not imply a worst-case sublinear running-time guarantee.

Algorithm 2 *Heuristic Local Algorithm (HLA)*

Input: A temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ-temporal motif $\mathcal{M}_\delta$, a source node s and two parameters α and ϵ
Output: A cluster C

1: $r(s) \leftarrow 1$; $r(u) \leftarrow 0$ for all $u \neq s$; $\pi(u) \leftarrow 0$ for all $u \in V$
2: $Q \leftarrow \{s\}$; $D \leftarrow \emptyset$; $G \leftarrow$ the de-temporal graph of $\mathcal{G}$; $\widehat{W} \leftarrow$ an empty directed sparse matrix
3: **while** $Q \neq \emptyset$ **do**
4: $\quad u \leftarrow Q.pop()$
5: $\quad$ **if** $u \notin D$ **then**
6: $\quad\quad D \leftarrow D \cup \{u\}$
7: $\quad\quad$ EFDEGREE(u)
8: $\quad$ PUSH(u)
9: **for** $u \in D$ and $EFD(u) \neq 0$ **do**
10: $\quad \pi(u) \leftarrow \pi(u)/EFD(u)$
11: $\pi_i \leftarrow$ the vertex with the i-th largest non-zero value in π
12: $C \leftarrow \arg\min \psi_{\widehat{W}}(C_i)$, where $C_i = \{\pi_1, \pi_2, \ldots, \pi_i\}$ and $\psi_{\widehat{W}}(C_i)$ is the edge-based conductance of C_i in $\widehat{W}$
13: **return** C
14: **function** EFDEGREE(u)
15: $\quad EFD(u) \leftarrow 0$
16: $\quad$ **for** each edge $(u, v) \in G$ **do**
17: $\quad\quad (\widehat{W})_{uv} \leftarrow$ the edge-focused weight $(W_{ef})_{uv}$ by Equation 1
18: $\quad\quad EFD(u) += (\widehat{W})_{uv}$
19: **function** PUSH(u)
20: $\quad$ **if** $EFD(u) > 0$ and $r(u) \geq \epsilon \cdot EFD(u)$ **then**
21: $\quad\quad$ **for** each $v \in \{w|(\widehat{W})_{uw} \neq 0\}$ **do**
22: $\quad\quad\quad r(v) \leftarrow r(v) + (1-\alpha)r(u) \cdot (\widehat{W})_{uv}/EFD(u)$
23: $\quad\quad\quad Q.push(v)$
24: $\quad\quad \pi(u) \leftarrow \pi(u) + \alpha r(u)$, $r(u) \leftarrow 0$

5 EXPERIMENTAL EVALUATION

5.1 Experimental Setup

Datasets. We evaluate our solutions on eight publicly-available datasets (Table 3), which are widely used benchmarks for temporal motif analysis and temporal graph clustering [19, 38, 49, 59–61, 64, 70, 74, 77]. Specifically, *CM* (CollegeMsg) contains message exchanges on a Facebook-like platform at UC Irvine. *EE* (email-Eu-core-temporal) consists of email interactions among users within a European research institution. *SA* (SMS-A) contains the short message interaction among users. *FW* (facebook-wall) is the wall posts between users on the social network Facebook located in the New Orleans region. *MO* (sx-mathoverflow), *UB* (sx-askubuntu), *SU* (sx-superuser), and *SO* (sx-stackoverflow) are temporal interaction networks between users on Math Overflow, Ask Ubuntu, Super User, and Stack Overflow, in terms of comments, questions, and answers. All datasets can be downloaded from https://networkrepository.com/ and https://snap.stanford.edu/data/. Note that the self-loops are deleted, and the largest weakly connected components of these datasets are used in the experiments.

Competitors. We compare our proposed *AGA* (Algorithm 1) and *HLA* (Algorithm 2) with ten representative baselines in three categories. (1) *Lower-order temporal graph clustering: TSC* [66], *TMO* [55], and *TDS* [58], which employ spectral clustering, modularity optimization, and density optimization, respectively. (2)

Table 3: Dataset statistics.

Dataset	$n = \|V\|$	$m = \|\mathcal{E}\|$	$\bar{m} = \|E\|$	Time Span
CM	1,893	59,794	20,292	193 days
EE	986	327,336	24,929	903 days
MO	24,668	390,348	227,933	2,350 days
SA	30,278	445,390	56,115	338 days
UB	152,599	723,628	541,751	2,613 days
FW	43,953	851,913	262,631	1,561 days
SU	189,191	1,106,530	852,335	2,773 days
SO	2,572,345	47,895,496	34,868,599	2,774 days

Higher-order static graph clustering: HSC [6, 30, 72], *HMO* [4, 29], and *HDS* [13, 26, 68], which apply higher-order spectral clustering, modularity optimization, and density optimization to the de-temporal graph, respectively. (3) *Higher-order temporal graph clustering: THDS* [64], *THSC* [6], *THLC* [78], and *THPR* [42]. THDS optimizes temporal motif density, while THSC, THLC, and THPR optimize temporal motif conductance (Definition 6). Existing efficient temporal-motif counting algorithms are often tailored to specific motif families [19, 38, 60, 77]. To uniformly support query motifs with different topologies and sizes, we employ the general-purpose chronological matcher of Mackey et al. [52] as the underlying matching routine for THSC, THLC, THPR, AGA, and HLA. For THSC, THLC, THPR, and AGA, motif matching is performed globally, with AGA immediately aggregating matched instances into edge-focused weights without materializing the complete instance collection. In contrast, HLA invokes the matcher on demand to compute the required edge-focused weights during local diffusion, avoiding global motif enumeration and weighted-graph construction. The theoretical cost $C(\mathcal{M}_\delta)$ in Table 1 is independent of the specific exact weight-computation routine and is instantiated by this Mackey-based implementation in our experiments, rather than by a specialized counting algorithm. When an exact edge-conditioned counting algorithm is available, it can replace this backend without changing the definition of W_{ef} or the subsequent clustering procedure.

Parameters. We evaluate six representative temporal motifs (Figure 4) with different topologies and sizes, following prior studies [19, 38, 49, 59, 60, 64, 70, 74, 77]. The full version [3] further discusses their practical applications. Following [38, 52, 59, 70], we use M_2 as the default query motif and set the duration threshold δ to one hour. Unless otherwise specified, we set $\alpha = 0.2$ and $\epsilon = 1/m$, following [2, 47, 75]. For the source-dependent methods THLC and HLA, we randomly sample 50 source vertices from the entire graph and report the average runtime and clustering quality over these queries, unless otherwise specified. For all other baselines, we use their default parameters.

5.2 Clustering Efficiency Evaluation

Our efficiency evaluation focuses on algorithms that directly optimize temporal motif conductance: *THSC*, *THLC*, *THPR*, *AGA*, and *HLA*. They share our objective and all require temporal motif computation. Other baselines (*TSC*, *TMO*, *TDS*, *HSC*, *HMO*, *HDS*, and *THDS*) optimize different objectives, so their runtimes are not directly comparable here. Nevertheless, we include all of them in the next section's quality evaluation for a broader empirical comparison.

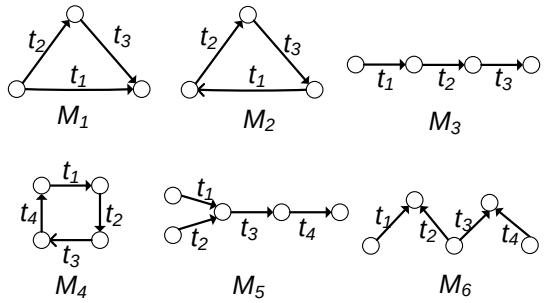
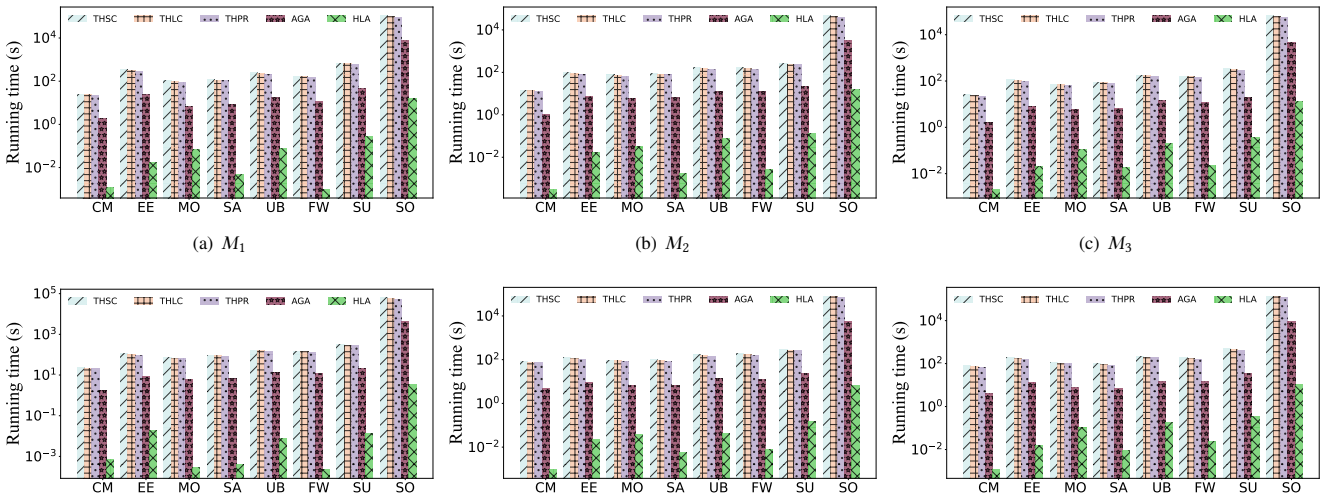


Figure 4: Temporal motifs used for experimental evaluation. $t_1 \le t_2 \le t_3 \le t_4$. The temporal duration δ is treated as a parameter.

Exp-1: Running time of different temporal motif conductance algorithms. Figure 5 shows the running time of *THSC*, *THLC*, *THPR*, *AGA*, and *HLA* with varying temporal motifs, with analogous trends observed across different δ (details in [3]). As can be seen, we have: (1) *HLA* is consistently faster than other methods. For example, on SO and M_6, *HLA* takes 10 seconds to obtain the result, while *AGA* *THSC*, *THLC*, and *THPR* require 9154, 139510, 126581,121248 seconds, respectively. A detailed runtime breakdown is provided in the full version [3]. Unlike alternatives, *HLA* does not precompute the weight $(W_{ef})uv$ for every edge (u, v); instead, it calculates $(Wef)_{uv}$ only upon accessing the edge (u, v). Consequently, *HLA*'s runtime mainly depends on the locally explored region rather than the total graph size. (2) *AGA* is the runner-up on all datasets and all temporal motifs. The efficiency of *AGA* can be attributed to its sparse edge-focused weighted graph representation and the reduced cost of subsequent clustering. Since *THSC*, *THLC*, *THPR*, and *AGA* use the same general-purpose temporal-motif matching backend, the observed speedups do not originate from a faster motif-discovery procedure. These results give preliminary evidence that the proposed solutions are highly efficient in practice.

Exp-2: Running time of *HLA* with varying parameters. Figure 6 illustrates how the parameters α and ϵ affect the running time of *HLA* with M_2 as the query temporal motif and $\delta = 1$ hour. Other query temporal motifs and δ can also obtain similar results. As can be seen, we have (1) the running time of *HLA* decreases with increasing α. An intuitive explanation is that when α increases, each vertex u that needs to transmit information will pass on a smaller residual amount to its neighbors (Line 22 of Algorithm 2), resulting in a lower probability for its neighbors to be executed in the next round of information transmission (Line 20 of Algorithm 2). Thus, when α increases, fewer nodes are visited, resulting in the locality of *HLA* being stronger (details in Algorithm 2). (2) The running time of *HLA* decreases with increasing ϵ. An intuitive explanation is that when ϵ increases, each vertex u has a smaller probability of being able to transmit information (Line 20 of Algorithm 2), resulting in fewer neighbors being visited and stronger locality (details in Algorithm 2). Thus, these results align with our theoretical analysis (Table 1).

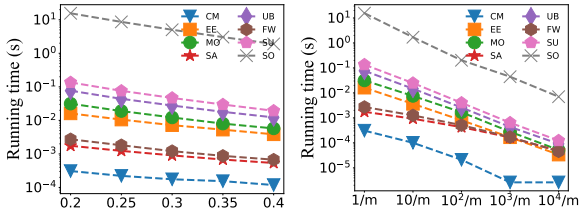
Exp-3: Scalability testing. We further investigated the scalability of our approaches using eight synthetically generated graphs. Figure 7 shows the runtime for query motif M_2 at $\delta = 1$ hour (other settings exhibited similar patterns). These graphs are produced by sampling edges or nodes from the SO dataset, which has the highest edge volume, at percentages from 50% to 100%. The results confirm



(a) M_1 (b) M_2 (c) M_3

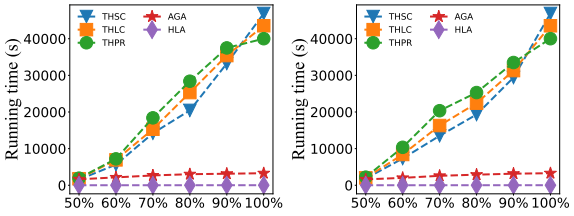
(d) M_4 (e) M_5 (f) M_6

Figure 5: Running time of different temporal motif conductance algorithms with varying temporal motif.



(a) Vary α (b) Vary ϵ

Figure 6: Running time of *HLA* with varying parameters.



(a) Varying n (b) Varying m

Figure 7: Scalability testing.

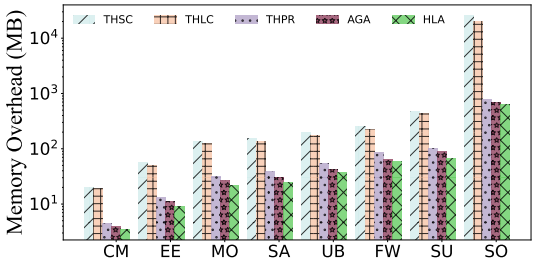


Figure 8: Memory overhead (excluding the graph itself).

that our algorithms, *AGA* and *HLA*, scale efficiently with increasing graph size. In contrast, the baselines show unstable performance, with runtimes that fluctuate significantly. These results demonstrate favorable scalability of our methods under the evaluated graph sizes and temporal-motif settings.

Exp-4: Memory overhead comparison. The memory consumption of all algorithms, evaluated with query motif M_2 and $\delta = 1$ hour, is compared in Figure 8; results for other parameter settings follow a consistent pattern. As theoretically predicted, our proposed algorithms, *AGA* and *HLA*, significantly reduce memory usage across all datasets, outperforming all baselines by up to an order of magnitude. A case in point is the SO dataset, where *AGA* and *HLA* use only 698 MB and 643 MB, respectively, in stark contrast to the substantial 25628 MB and 20226 MB required by *THSC* and *THLC*. This efficiency originates from our edge-focused weighted graph projection, which confines memory complexity to $O(m)$ by assigning weights solely to the original edges. Conversely, *THSC* and *THLC* incur a much higher cost of $O(\min n^2, \binom{k}{2} num(\mathcal{M}_\delta))$ due to their construction of hyperedges for motif instances. Furthermore, *HLA* proves slightly more efficient than *AGA* and *THPR*. This is because *THPR* maintains some auxiliary structures, such as temporal motif resident information for each vertex (Definition 8), and *AGA* requires additional space to store the intermediate variables $(W_{ef})(u, v)$ for all edge (u, v); instead, *HLA* only store $(Wef)_{uv}$ upon accessing the edge (u, v). These results are consistent with the storage benefits of the sparse edge-focused representation. A detailed memory breakdown is provided in the full version [3].

5.3 Clustering Quality Evaluation

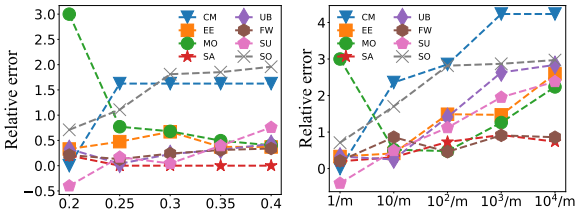
Evaluating the quality of temporal clustering poses greater challenges than its static counterpart, primarily due to the absence of ground-truth clusters for temporal networks [47, 61, 66]. Fortunately, we can adopt the following two widely used temporal structure metrics [45–47, 61, 66, 83] for qualitative testing: temporal density (*TD*) and temporal conductance (*TC*). Specifically, let C be the identified cluster, the two metrics are defined as follows: $TD(C) = |\{(u, v, t) \in \mathcal{E} | u, v \in C\}|/|C|$ and $TC(C) = |\{(u, v, t) \in \mathcal{E} | u \in C, v \in \bar{C}\}|/\sum_{u \in C} |\{(u, v, t) \in \mathcal{E}\}|$. Clearly, *TD* computes the temporal density of the internal structure of the cluster, *TC* measures the

Table 4: Clustering quality and output cluster sizes of different methods. Lower TMC and TC values and higher TD values indicate better performance under the corresponding metrics. The best and second-best metric values are marked in **bold** and underlined, respectively. Cluster size is reported for context and is not used for ranking. For Size, k denotes 10^3. OOT indicates that a method fails to finish within two days.

TMC/TC/TD/Size	CM	EE	MO	SA	UB	FW	SU	SO
TSC	0.60/**0.20**/2.66/112	0.45/**0.20**/279.82/95	0.87/**0.17**/2.80/850	0.81/0.41/24.07/460	0.69/**0.03**/2.54/2.4k	0.58/**0.02**/227.00/1.7k	0.97/**0.02**/6.85/3.6k	0.83/**0.07**/2.00/48k
TMO	0.56/0.85/28.71/250	0.29/0.42/656.38/220	0.75/0.31/6.99/4.5k	0.64/**0.01**/59.63/3.6k	0.71/0.28/3.16/12k	0.52/0.19/29.03/6.8k	0.70/0.23/2.52/15.5k	0.61/0.23/2.95/130k
TDS	0.67/0.83/**243.61**/31	0.54/0.43/**4878.00**/30	0.70/0.51/**547.89**/250	0.69/**0.01**/**6063.00**/50	0.71/0.55/**119.09**/2k	0.52/0.24/197.68/2.7k	0.69/0.53/**153.47**/2.5k	0.67/0.49/**535.03**/30k
HSC	0.71/0.89/9.03/85	0.79/0.66/10.46/66	0.82/0.89/52.68/520	0.68/0.69/23.58/480	0.95/0.81/64.79/1.9k	0.96/0.47/163.00/960	0.91/0.46/46.82/2.3k	OOT
HMO	0.74/0.88/2.34/690	0.75/0.72/112.30/380	0.72/0.88/56.11/650	0.77/0.59/21.06/3.2k	0.94/0.73/114.06/1.4k	0.72/0.52/35.11/6.1k	0.79/0.76/30.04/7k	OOT
HDS	0.76/0.88/30.21/240	0.72/0.67/45.18/190	0.59/0.86/117.35/340	0.87/0.71/47.08/1.1k	0.95/0.66/67.08/3k	0.95/0.67/114.15/2.4k	0.92/0.73/36.56/6.9k	OOT
THDS	0.10/0.85/44.22/64	0.13/0.40/801.33/52	0.22/0.90/229.60/130	0.68/0.47/203.00/240	0.24/0.92/45.15/360	0.50/0.39/**890.66**/190	0.27/0.92/32.71/620	0.30/0.99/16.33/120
THSC	**0.01**/0.52/37.46/13	0.08/0.39/802.00/31	0.09/0.98/17.50/90	0.95/0.47/213.00/135	0.09/0.92/44.61/62	0.66/0.79/17.33/79	0.12/0.90/61.72/415	0.15/0.96/9.28/18
THLC	**0.01**/0.85/38.28/16	0.05/0.36/139.77/34	0.12/0.87/150.33/110	0.95/0.46/190.00/142	0.06/0.92/47.01/68	0.64/0.52/83.20/84	0.17/0.95/22.78/390	0.32/0.80/115.46/21
THPR	0.25/0.83/28.50/44	0.06/0.38/304.06/36	0.12/0.95/51.48/125	0.95/0.91/20.00/150	0.06/0.91/42.86/73	0.66/0.79/17.33/91	0.05/0.97/11.41/360	0.22/0.85/16.66/33
AGA	**0.01**/0.52/47.46/15	**0.03**/0.34/805.00/26	**0.03**/0.48/270.50/85	**0.50**/0.37/223.00/120	**0.03**/0.12/44.61/55	**0.33**/0.09/227.33/74	0.05/0.20/63.72/403	**0.07**/0.16/159.28/14
HLA	**0.01**/0.86/20.70/17	0.04/0.39/250.12/29	0.12/0.75/260.31/94	0.60/0.42/162.85/136	0.04/0.85/92.47/61	0.40/0.15/199.31/82	**0.03**/0.80/49.54/388	0.12/0.51/259.83/18

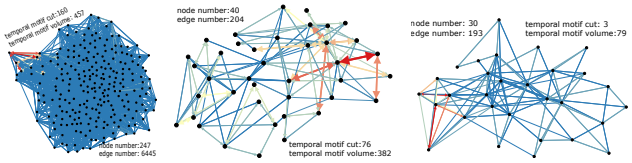
temporal separability of the cluster. Thus, the larger $TD(C)$ (resp., smaller $TC(C)$) indicates the denser inside (resp., sparser outside) of C. Besides, we also evaluate the clustering quality using the proposed temporal motif conductance (*TMC*), as defined in Definition 6. As a consequence, smaller *TMC* and *TC* indicate higher clustering quality, while larger *TD* indicates higher clustering quality. It is worth noting that these metrics capture complementary aspects of temporal clustering quality. *TMC* evaluates higher-order temporal motif preservation, which is the objective studied in this paper. *TC* and *TD*, on the other hand, are lower-order temporal edge-level metrics that respectively measure external temporal separability and internal temporal density. Our algorithms do not directly optimize *TC* or *TD*. Therefore, their performance on *TC* and *TD* serves as an additional structural validation rather than a direct consequence of optimizing the proposed objective.

Exp-5: Clustering quality of various clustering methods. Since traditional lower-order temporal (or higher-order static) graph clustering methods (i.e., *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, and *HDS*) do not rely on temporal motifs, Table 4 only shows results for M_2 as the query temporal motif and δ =1 hour. The empirical results of other settings are placed in the [3] due to space limitations. For the *TMC* metric, we have (1) *AGA* outperforms all other methods on seven out of eight datasets. These results indicate that *AGA* effectively optimizes the proposed TMC objective under the evaluated settings. (2) *HLA* is superior to all the other methods except *AGA*. This is because *HLA* builds upon the edge-focused counting principle of *AGA*; it maintains strong practical performance despite lacking a theoretical guarantee. (3) *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, and *HDS* obtain less favorable TMC values. This is because they ignore the crucial temporal information or higher-order information. For the *TC&TD* metric, we have: (1) the proposed algorithm *AGA* achieves the best overall performance without the situation where one metric is excellent but another is poor. (2) *TSC* has good *TC* value but has poor *TD* value. On the contrary, *TDS* has good *TD* value but poor *TC* value. This is because *TSC* merely considers the external temporal separateness but ignores the internal temporal density, *TDS* merely considers the internal temporal density but ignores the external temporal separateness. (3) Other methods have the worst performance. This is because either they are static methods, which fail to take into account temporal information, or they do not consider the temporal dense inside or the temporal sparse outside.



(a) Relative error with varying α (b) Relative error with varying ϵ

Figure 9: Quality comparison between *AGA* and *HLA*.



(a) *TDS* (b) *THDS* (c) Our model *MgcCT*

Figure 10: Case studies. The thicker and darker the edge, the more temporal cycles it participates in.

To assess cluster recovery beyond the optimized TMC objective, we further evaluate F1 and NMI on synthetic temporal graphs with known planted-cluster membership. This evaluation provides complementary recovery evidence under controlled motif-aligned settings. The complete protocol and results are provided in [3].

Exp-6: Quality comparison between *AGA* and *HLA*. Figure 9 compares the clustering quality of the approximate global algorithm (*AGA*) against the heuristic local algorithm (*HLA*). Using the temporal motif conductance ρ_{aga} from *AGA* as a benchmark, we define the relative error (RE) of *HLA* as $RE = (\rho_{hla} - \rho_{aga})/\rho_{aga}$. As can be seen, we have (1) the RE ranges approximately from -0.5 to 4, showing that the quality difference between *HLA* and *AGA* can be substantial for some parameter settings . (2) The RE generally increases with parameters α and ϵ. This trend is attributed to the role of these parameters in limiting the exploration scope of *HLA*; higher values restrict the search to a narrower neighborhood, increasing the

risk of converging to a suboptimal local solution. Thus, the heuristic local algorithm *HLA* can identify high-quality clusters in practice.

Exp-7: Case studies. We conduct case studies on the Bitcoin dataset [35], which comprises 3,775 nodes and 24,180 temporal edges. Here, nodes represent users and temporal edges indicate when the corresponding users conducted the transaction through the well-known Bitcoin Alpha platform. To demonstrate the utility of our model, we select the temporal cycle M_4 in Figure 4 as the query motif, as it is a recognized indicator for identifying money laundering activities [34, 37, 67]. Figure 10 visualizes the clusters identified by each method. Note that clusters detected by *TMO*, *TSC*, *HMO*, *HSC*, and *HDS* are either excessively large or too small for effective visualization. For instance, clusters identified by *TMO* and *HMO* contain over 1,000 vertices, reflecting the modularity resolution limitation [15]. Furthermore, Table 4 confirms their poor performance in this context. In this case, *TDS* identifies a cluster of 247 users and 6,445 edges, densely connected via dyadic temporal edges but lacking higher-order temporal cycles (Figure 10(a)). Consequently, the number of temporal cycle instances is an order of magnitude lower than the number of edges. A detailed inspection of Figure 10(a) reveals that *TDS* mainly captures a temporally dense dyadic-edge region, but it does not concentrate temporal-cycle structures well. Although *THDS* identifies more temporal-cycle instances inside the cluster (Figure 10(b)), the resulting cluster also has many temporal-cycle connections to outside vertices, violating the external sparsity requirement and leading to a less separated result [25, 31, 36, 41, 51]. In contrast, the cluster identified by our model *MgcCT* in Figure 10(c) achieves a better balance between internal temporal-cycle preservation and external temporal-cycle separability. Specifically, it preserves suspicious temporal-cycle structures inside the cluster while keeping the number of cross-cluster temporal cycles small. This suggests that *MgcCT* can reveal compact temporal-cycle patterns that are structurally consistent with suspicious circulation behaviors [34, 48, 77].

5.4 Other Experiments

Due to space limitations, additional evaluations of runtime and clustering effectiveness under varying parameter settings, ground-truth cluster recovery, source-node sensitivity, locality, and runtime and memory breakdowns are provided in the full version [3].

6 RELATED WORK

Static Graph Clustering has received much attention over past decades [16, 36, 53]. Representative models include edge cut based (e.g., ratio cut or normalized cut [73]), null model based (e.g., modularity [56]), and subgraph cohesiveness based (e.g., clique or k-core [10]). The above methods mainly focus on the internal or external dyadic edges of the cluster, resulting in that cannot capture the more informative and semantic higher-order structure. Until recently, motifs (i.e., small subgraphs) have been shown to be indispensable for modeling and understanding the higher-order organization of complex networks [6]. Thus, a wide spectrum of motif-based higher-order graph clustering methods have been proposed [4–6, 13, 29, 30, 68, 72, 76, 78, 82]. For example, motif modularity was proposed to extend the traditional modularity by optimizing the difference between the fraction of motif instances within the cluster and the fraction in a random network preserving the same degree of vertices [4, 29]. Higher-order densest subgraph model was proposed where the density is defined as the number of motif instances divided by the size of vertices [13, 26, 68]. Theoretical guarantees have also been studied for hypergraph clustering. For example, Li and Milenkovic [39] established approximation guarantees for hypergraph partitioning with submodular hyperedge cut costs, while Takai et al. [69] developed PageRank-based hypergraph clustering algorithms with conductance guarantees. These studies provide theoretical foundations for higher-order clustering beyond three-node motifs. However, these methods focus on static structures and do not directly account for the temporal ordering and duration constraints considered in our problem.

Temporal Graph Clustering introduces a key temporal dimension into graph clustering [11, 55, 58, 66]. For example, Silva et al. [66] generalized sparsest and normalized cuts to temporal graphs by introducing temporal smoothness constraints on cut evolution. Mucha et al. [55] developed temporal modularity to quantify clustering quality across time slices, rewarding stable partitions with high instantaneous modularity and penalizing vertex migrations between clusters. Qin et al. [61] investigated bursting clusters in temporal networks, where clusters exhibit transient density peaks. Yu et al. [79] proposed an index-based method with pruning strategies to accelerate historical k-core discovery. However, these approaches are fundamentally limited by their focus on low-order connectivity patterns, rendering them inapplicable to our problem domain. Recently, Fu et al. [17, 18] studied the higher-order graph clustering on dynamic graphs, which aims to maintain the new higher-order clusters when vertices/edges are inserted/deleted. Their methods focus on dynamic graph updates or snapshot-level structures, while our model explicitly considers temporally ordered motif instances within a duration threshold. As a result, these dynamic higher-order graph clustering methods are orthogonal to ours [20, 47, 58, 61, 64].

7 CONCLUSION

To capture the interplay of continuous topological-temporal dependencies and higher-order structural patterns in temporal clusters, we introduce a novel metric, temporal motif conductance, quantifying cluster cohesion through motif-constrained random walk dynamics. We formalize our objective as minimizing this conductance and rigorously establish its NP-hardness. To address this computational challenge, we propose two complementary algorithms: a global approximation algorithm *AGA* based on a sparse edge-focused weighted graph with a Cheeger-type guarantee, and a heuristic local algorithm *HLA* that computes edge-focused weights on demand without globally constructing the weighted graph in advance, thereby improving the trade-off between runtime efficiency and clustering quality. Extensive experiments on eight real-world temporal networks demonstrate that our framework outperforms ten competitors in both clustering efficiency and quality.

In this paper, we consider an offline setting in which the observed temporal graph is available before clustering. Online maintenance under streaming edge updates is beyond the scope of this work. Supporting incremental cluster maintenance over temporal edge streams remains an interesting direction for future work.

REFERENCES

[1] Noga Alon and Vitali D Milman. 1985. λ_1, isoperimetric inequalities for graphs, and superconcentrators. *Journal of Combinatorial Theory, Series B* 38, 1 (1985), 73–88.

[2] Reid Andersen, Fan R. K. Chung, and Kevin J. Lang. 2006. Local Graph Partitioning using PageRank Vectors. In *FOCS*. 475–486.

[3] Anonymous. 2026. full version available at: https://github.com/longlonglin/MgcCT. (2026).

[4] Alex Arenas, Alberto Fernandez, Santo Fortunato, and Sergio Gomez. 2008. Motif-based communities in complex networks. *Journal of Physics A: Mathematical and Theoretical* 41, 22 (2008), 224001.

[5] Austin R. Benson, David F. Gleich, and Jure Leskovec. 2015. Tensor Spectral Clustering for Partitioning Higher-order Network Structures. In *SDM*. 118–126.

[6] Austin R Benson, David F Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.

[7] Filippo Maria Bianchi, Daniele Grattarola, and Cesare Alippi. 2020. Spectral Clustering with Graph Neural Networks for Graph Pooling. In *ICML*. 874–883.

[8] Xin-Wei Cai, Xiangyu Ke, Kai Wang, Lu Chen, Tianming Zhang, Qing Liu, and Yunjun Gao. 2023. Efficient Temporal Butterfly Counting and Enumeration on Temporal Bipartite Graphs. *Proc. VLDB Endow.* 17, 4 (2023), 657–670.

[9] Abdulkadir Çelikkanat, Nikolaos Nakis, and Morten Mørup. 2024. Continuous-Time Graph Representation with Sequential Survival Process. In *AAAI*. 11177–11185.

[10] Lijun Chang and Lu Qin. 2019. Cohesive Subgraph Computation Over Large Sparse Graphs. In *Proceedings of ICDE*. 2068–2071.

[11] Yun Chi, Xiaodan Song, Dengyong Zhou, Koji Hino, and Belle L. Tseng. 2007. Evolutionary spectral clustering by incorporating temporal smoothness. In *KDD*. 153–162.

[12] Karen D. Devine, Erik G. Boman, and George Karypis. 2006. Partitioning and Load Balancing for Emerging Parallel Applications and Architectures. In *Parallel Processing for Scientific Computing*. 99–126.

[13] Yixiang Fang, Kaiqiang Yu, Reynold Cheng, Laks V. S. Lakshmanan, and Xuemin Lin. 2019. Efficient Algorithms for Densest Subgraph Discovery. *Proc. VLDB Endow.* 12, 11 (2019), 1719–1732.

[14] Miroslav Fiedler. 1973. Algebraic connectivity of graphs. *Czechoslovak mathematical journal* 23, 2 (1973), 298–305.

[15] Santo Fortunato and Marc Barthelemy. 2007. Resolution limit in community detection. *Proceedings of the national academy of sciences* 104, 1 (2007), 36–41.

[16] Santo Fortunato and Darko Hric. 2016. Community detection in networks: A user guide. *CoRR* abs/1608.00163 (2016).

[17] Dongqi Fu, Dawei Zhou, and Jingrui He. 2020. Local Motif Clustering on Time-Evolving Graphs. In *SIGKDD*. 390–400.

[18] Dongqi Fu, Dawei Zhou, Ross Maciejewski, Arie Croitoru, Marcus Boyd, and Jingrui He. 2023. Fairness-Aware Clique-Preserving Spectral Clustering of Temporal Graphs. In *WWW*. 3755–3765.

[19] Zhongqiang Gao, Chuanqi Cheng, Yanwei Yu, Lei Cao, Chao Huang, and Junyu Dong. 2022. Scalable motif counting for large-scale temporal graphs. In *ICDE*. 2656–2668.

[20] Aristides Gionis, Lutz Oettershagen, and Ilie Sarpe. 2024. Mining Temporal Networks. In *WWW*. 1260–1263.

[21] David F. Gleich and C. Seshadhri. 2012. Vertex neighborhoods, low conductance cuts, and good seeds for local community methods. In *KDD*. 597–605.

[22] Gene H Golub and Charles F Van Loan. 2013. *Matrix computations*. JHU press.

[23] Saket Gurukar, Sayan Ranu, and Balaraman Ravindran. 2015. COMMIT: A Scalable Approach to Mining Communication Motifs from Dynamic Networks. In *SIGMOD*. 475–489.

[24] Liangzhe Han, Xiaojian Ma, Leilei Sun, Bowen Du, Yanjie Fu, Weifeng Lv, and Hui Xiong. 2022. Continuous-Time and Multi-Level Graph Representation Learning for Origin-Destination Demand Prediction. In *KDD*. 516–524.

[25] Yue He, Longlong Lin, Pingpeng Yuan, Ronghua Li, Tao Jia, and Zeli Wang. 2024. CCSS: Towards conductance-based community search with size constraints. *Expert Syst. Appl.* 250 (2024), 123915.

[26] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2023. Scaling Up k-Clique Densest Subgraph Detection. *Proc. ACM Manag. Data* 1, 1 (2023), 69:1–69:26.

[27] Petter Holme. 2015. Modern temporal network theory: A colloquium. *CoRR* abs/1508.01303 (2015).

[28] Petter Holme and Jari Saramaki. 2012. Temporal networks. *Physics Reports* 519 (2012), 97–125.

[29] Ling Huang, Hong-Yang Chao, and Guangqiang Xie. 2020. MuMod: A Micro-Unit Connection Approach for Hybrid-Order Community Detection. In *AAAI*. 107–114.

[30] Shixun Huang, Yuchen Li, Zhifeng Bao, and Zhao Li. 2021. Towards Efficient Motif-based Graph Partitioning: An Adaptive Sampling Approach. In *ICDE*. 528–539.

[31] Raj Kamal and Amitabha Bagchi. 2024. A Lovász-Simonovits Theorem for Hypergraphs with Application to Local Clustering. *Proc. ACM Manag. Data* 2, 4 (2024), 190:1–190:27.

[32] Lauri Kovanen, Márton Karsai, Kimmo Kaski, János Kertész, and Jari Saramäki. 2011. Temporal motifs in time-dependent networks. *Journal of Statistical Mechanics: Theory and Experiment* 2011, 11 (2011), P11005.

[33] Lauri Kovanen, Kimmo Kaski, János Kertész, and Jari Saramäki. 2013. Temporal motifs reveal homophily, gender-specific patterns, and group talk in call sequences. *Proceedings of the National Academy of Sciences* 110, 45 (2013), 18070–18075.

[34] Rohit Kumar and Toon Calders. 2018. 2SCENT: An Efficient Algorithm to Enumerate All Simple Temporal Cycles. *Proc. VLDB Endow.* 11, 11 (2018), 1441–1453.

[35] Srijan Kumar, Bryan Hooi, Disha Makhija, Mohit Kumar, Christos Faloutsos, and VS Subrahmanian. 2018. Rev2: Fraudulent user prediction in rating platforms. In *WSDM*. 333–341.

[36] Jure Leskovec, Kevin J. Lang, and Michael W. Mahoney. 2010. Empirical comparison of algorithms for network community detection. In *WWW*. 631–640.

[37] Michael Levi and Peter Reuter. 2006. Money laundering. *Crime and justice* 34, 1 (2006), 289–375.

[38] Jiantao Li, Jianpeng Qi, Yueling Huang, Lei Cao, Yanwei Yu, and Junyu Dong. 2024. MoTTo: Scalable Motif Counting with Time-aware Topology Constraint for Large-scale Temporal Graphs. In *CIKM*. 1195–1204.

[39] Pan Li and Olgica Milenkovic. 2017. Inhomogeneous Hypergraph Clustering with Applications. In *Advances in Neural Information Processing Systems 30*. 2308–2318. https://proceedings.neurips.cc/paper_files/paper/2017/hash/a50abba8132a77191791390c3eb19fe7-Abstract.html

[40] Zhiyuan Li, Xun Jian, Yue Wang, and Lei Chen. 2022. CC-GNN: A Community and Contraction-based Graph Neural Network. In *ICDM*, Xingquan Zhu, Sanjay Ranka, My T. Thai, Takashi Washio, and Xindong Wu (Eds.). 231–240.

[41] Longlong Lin, Yue He, Wei Chen, Pingpeng Yuan, Rong-Hua Li, and Tao Jia. 2025. Effective and Efficient Conductance-Based Community Search at Billion Scale. *IEEE Trans. Big Data* 11, 6 (2025), 3170–3184.

[42] Longlong Lin, Tao Jia, Zeli Wang, Jin Zhao, and Rong-Hua Li. 2024. PSMC: Provable and Scalable Algorithms for Motif Conductance Based Graph Clustering. In *KDD*. 1793–1803.

[43] Longlong Lin, Ronghua Li, and Tao Jia. 2023. Scalable and Effective Conductance-Based Graph Clustering. In *AAAI*. 4471–4478.

[44] Longlong Lin, Zeli Wang, Rong-Hua Li, Qiyu Liu, Hongchao Qin, and Jin Zhao. 2025. Provable Higher-Order Graph Clustering: The Power of Peeling-Based Approaches. *IEEE Trans. Knowl. Data Eng.* 37, 9 (2025), 5568–5583.

[45] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, and Hai Jin. 2022. Mining Diversified Top-rr Lasting Cohesive Subgraphs on Temporal Networks. *IEEE Trans. Big Data* 8, 6 (2022), 1537–1549.

[46] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Jifei Wang, Ling Liu, and Hai Jin. 2022. Mining Stable Quasi-Cliques on Temporal Networks. *IEEE Trans. Syst. Man Cybern. Syst.* 52, 6 (2022), 3731–3745.

[47] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Chun-Xue Zhu, Hongchao Qin, Hai Jin, and Tao Jia. 2024. QTCS: Efficient Query-Centered Temporal Community Search. *Proc. VLDB Endow.* 17, 6 (2024), 1187–1199.

[48] Penghang Liu, Rupam Acharyya, Robert E. Tillman, Shunya Kimura, Naoki Masuda, and Ahmet Erdem Sariyuce. 2023. Temporal Motifs for Financial Networks: A Study on Mercari, JPMC, and Venmo Platforms. *CoRR* abs/2301.07791 (2023).

[49] Paul Liu, Austin R. Benson, and Moses Charikar. 2019. Sampling Methods for Counting Temporal Motifs. In *WSDM*. 294–302.

[50] Penghang Liu, Valerio Guarrasi, and Ahmet Erdem Sariyüce. 2023. Temporal Network Motifs: Models, Limitations, Evaluation. *IEEE Trans. Knowl. Data Eng.* 35, 1 (2023), 945–957.

[51] Yunkai Lou, Chaokun Wang, Tiankai Gu, Hao Feng, Jun Chen, and Jeffrey Xu Yu. 2021. Time-Topology Analysis. *Proc. VLDB Endow.* 14, 13 (2021), 3322–3334.

[52] Patrick Mackey, Katherine Porterfield, Erin Fitzhenry, Sutanay Choudhury, and George Chin. 2018. A chronological edge-driven approach to temporal subgraph isomorphism. In *IEEE BigData*. 3972–3979.

[53] Markus Maier, Ulrike von Luxburg, and Matthias Hein. 2008. Influence of graph construction on graph-based clustering measures. In *NIPS*. 1025–1032.

[54] Marina Meila and Jianbo Shi. 2001. A Random Walks View of Spectral Segmentation. In *AISTATS*, Thomas S. Richardson and Tommi S. Jaakkola (Eds.).

[55] Peter J Mucha, Thomas Richardson, Kevin Macon, Mason A Porter, and Jukka-Pekka Onnela. 2010. Community structure in time-dependent, multiscale, and multiplex networks. *science* 328, 5980 (2010), 876–878.

[56] Mark EJ Newman. 2004. Fast algorithm for detecting community structure in networks. *Physical review E* 69, 6 (2004), 066133.

[57] Giang Hoang Nguyen, John Boaz Lee, Ryan A. Rossi, Nesreen K. Ahmed, Eunyee Koh, and Sungchul Kim. 2018. Continuous-Time Dynamic Network Embeddings. In *WWW*. 969–976.

[58] Lutz Oettershagen, Athanasios L. Konstantinidis, and Giuseppe F. Italiano. 2025. An Edge-Based Decomposition Framework for Temporal Networks. In *WSDM*. 735–743.

[59] Ashwin Paranjape, Austin R. Benson, and Jure Leskovec. 2017. Motifs in Temporal Networks. In *WSDM*. 601–610.

[60] Noujan Pashanasangi and C Seshadhri. 2021. Faster and generalized temporal triangle counting, via degeneracy ordering. In *KDD*. 1319–1328.

[61] Hongchao Qin, Ronghua Li, Ye Yuan, Guoren Wang, Lu Qin, and Zhiwei Zhang. 2022. Mining Bursting Core in Large Temporal Graph. *Proc. VLDB Endow.* 15, 13 (2022), 3911–3923.

[62] Ilie Sarpe and Fabio Vandin. 2021. odeN: Simultaneous Approximation of Multiple Motif Counts in Large Temporal Networks. In *CIKM*. 1568–1577.

[63] Ilie Sarpe and Fabio Vandin. 2021. Presto: Simple and scalable sampling techniques for the rigorous approximation of temporal motif counts. In *SDM*. 145–153.

[64] Ilie Sarpe, Fabio Vandin, and Aristides Gionis. 2024. Scalable Temporal Motif Densest Subnetwork Discovery. In *KDD*. 2536–2547.

[65] Jianbo Shi and Jitendra Malik. 1997. Normalized Cuts and Image Segmentation. In *CVPR*. 731–737.

[66] Arlei Silva, Ambuj K. Singh, and Ananthram Swami. 2018. Spectral Algorithms for Temporal Graph Cuts. In *WWW*. 519–528.

[67] Tiziano Squartini, Iman van Lelyveld, and Diego Garlaschelli. 2013. Early-warning signals of topological collapse in interbank networks. *Scientific Reports* 3 (2013).

[68] Bintao Sun, Maximilien Danisch, T.-H. Hubert Chan, and Mauro Sozio. 2020. KClist++: A Simple Algorithm for Finding k-Clique Densest Subgraphs in Large Graphs. *Proc. VLDB Endow.* 13, 10 (2020), 1628–1640.

[69] Yuuki Takai, Atsushi Miyauchi, Masahiro Ikeda, and Yuichi Yoshida. 2020. Hypergraph Clustering Based on PageRank. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1970–1978. https://doi.org/10.1145/3394486.3403248

[70] Nishil Talati, Haojie Ye, Sanketh Vedula, Kuan-Yu Chen, Yuhan Chen, Daniel Liu, Yichao Yuan, David Blaauw, Alex Bronstein, Trevor Mudge, et al. 2022. Mint: An accelerator for mining temporal motifs. In *MICRO*. 1270–1287.

[71] David Tolliver and Gary L. Miller. 2006. Graph Partitioning by Spectral Rounding: Applications in Image Segmentation and Clustering. In *CVPR*. 1053–1060.

[72] Charalampos E. Tsourakakis, Jakub Pachocki, and Michael Mitzenmacher. 2017. Scalable Motif-aware Graph Clustering. In *WWW*. 1451–1460.

[73] Ulrike Von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and computing* 17, 4 (2007), 395–416.

[74] Jingjing Wang, Yanhao Wang, Wenjun Jiang, Yuchen Li, and Kian-Lee Tan. 2020. Efficient Sampling Algorithms for Approximate Temporal Motif Counting. In *CIKM*. 1505–1514.

[75] Zhewei Wei, Xiaodong He, Xiaokui Xiao, Sibo Wang, Shuo Shang, and Ji-Rong Wen. 2018. TopPPR: Top-k Personalized PageRank Queries with Precision Guarantees on Large Graphs. In *SIGMOD*. 441–456.

[76] Tao Wu, Austin R. Benson, and David F. Gleich. 2016. General Tensor Spectral Co-clustering for Higher-Order Data. In *NIPS*, Daniel D. Lee, Masashi Sugiyama, Ulrike von Luxburg, Isabelle Guyon, and Roman Garnett (Eds.). 2559–2567.

[77] Yuyang Xia, Yixiang Fang, and Wensheng Luo. 2025. Efficiently Counting Triangles in Large Temporal Graphs. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.

[78] Hao Yin, Austin R. Benson, Jure Leskovec, and David F. Gleich. 2017. Local Higher-Order Graph Clustering. In *KDD*. 555–564.

[79] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On Querying Historical K-Cores. *Proc. VLDB Endow.* 14, 11 (2021), 2033–2045.

[80] Yichao Yuan, Haojie Ye, Sanketh Vedula, Wynn Kaza, and Nishil Talati. 2023. Everest: GPU-Accelerated System For Mining Temporal Motifs. *Proc. VLDB Endow.* 17, 2 (2023), 162–174.

[81] Yifei Zhang, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Significant Engagement Community Search on Temporal Networks. In *DASFAA*. 250–258.

[82] Dawei Zhou, Si Zhang, Mehmet Yigit Yildirim, Scott Alcorn, Hanghang Tong, Hasan Davulcu, and Jingrui He. 2017. A Local Algorithm for Structure-Preserving Graph Cut. In *KDD*. 655–664.

[83] Chun-Xue Zhu, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Discovering Cohesive Temporal Subgraphs with Temporal Density Aware Exploration. *J. Comput. Sci. Technol.* 37, 5 (2022), 1068–1085.

A APPENDIX

A.1 Running Example of Difference Weighted Graph Representations

Consider the four-node directed temporal path motif M_3 in Fig. 4 with $\delta = 4$. Following the edge directions and nondecreasing temporal ordering required by Definition 2, we can identify six valid δ-instances in Fig. 1(a), which fall into two groups: $\mathcal{I}_{a,b} = \{(v_1, v_2, a), (v_2, v_5, 4), (v_5, v_4, b)\}, a \in \{2, 3\}, b \in \{5, 6\}$, $\mathcal{J}_b = \{(v_2, v_5, 4), (v_5, v_4, b), (v_4, v_6, 6)\}, b \in \{5, 6\}$. The first group contains four instances with vertex set $\{v_1, v_2, v_5, v_4\}$, while the second contains two instances with vertex set $\{v_2, v_5, v_4, v_6\}$. All six instances satisfy the required temporal ordering and duration constraints. In the conventional pairwise motif-weighted representation, each four-node instance contributes to all $\binom{4}{2} = 6$ pairs of co-occurring vertices. Since the two groups share three vertices $\{v_2, v_5, v_4\}$, their contributions involve $2\binom{4}{2} - \binom{3}{2} = 9$ distinct vertex pairs. More specifically, the corresponding pairwise weights are

$$A_{v_1v_2} = A_{v_1v_5} = A_{v_1v_4} = 4,$$
$$A_{v_2v_5} = A_{v_2v_4} = A_{v_5v_4} = 6,$$
$$A_{v_2v_6} = A_{v_5v_6} = A_{v_4v_6} = 2.$$

Thus, the conventional projection introduces weighted connections between vertices that co-occur in a motif instance, even when they are not directly connected in the original de-temporal graph. For example, $\{v_1, v_4\}$ and $\{v_2, v_6\}$ receive positive weights despite having no corresponding original edges.

In contrast, the proposed edge-focused representation assigns motif contributions only to original de-temporal edges that participate in valid instances. For all six instances, only four directed edges receive nonzero weights:

$$(W_{ef})_{v_1v_2} = 4, \qquad (W_{ef})_{v_2v_5} = 6,$$
$$(W_{ef})_{v_5v_4} = 6, \qquad (W_{ef})_{v_4v_6} = 2.$$

(v_1, v_2) participates in four instances, (v_2, v_5) and (v_5, v_4) each participate in six instances, and (v_4, v_6) participates in two instances. All other edge-focused weights are zero for this query motif.

Consequently, the conventional pairwise projection contains nine distinct weighted connections, whereas the edge-focused projection requires only four original-edge-supported connections. Notably, even though (v_2, v_4) exists in the original de-temporal graph, it does not participate in any of these valid M_3 instances and hence receives zero edge-focused weight, while its pairwise co-occurrence weight is 6. This example highlights the key distinction between the two representations: conventional pairwise projection connects vertices according to motif co-occurrence, whereas our edge-focused projection aggregates temporal-motif participation on the original edges that actually support the instances. By restricting weighted connections to the original edge support, the proposed representation avoids additional motif-induced pairwise links and can reduce the storage and subsequent partitioning overhead when the conventional pairwise projection becomes substantially denser.

A.2 Cheeger Guarantee for the Standard Pairwise Motif Projection

Let $\mathcal{A}$ be the standard zero-diagonal pairwise motif-weighted matrix, and consider a k-vertex temporal motif instance with i vertices in C and $k - i$ in $\bar{C}$. If the instance crosses the cluster boundary, it contributes $i(k - i)$ to $cut_{\mathcal{A}}(C)$. Since $k - 1 \le i(k - i) \le \left\lfloor \frac{k^2}{4} \right\rfloor$, summing over all crossing instances yields $(k - 1)cut_{\mathcal{M}_\delta}(C) \le cut_{\mathcal{A}}(C) \le \left\lfloor \frac{k^2}{4} \right\rfloor cut_{\mathcal{M}_\delta}(C)$. Each motif instance contributes exactly $k - 1$ to the weighted degree of each participating vertex. Therefore, $vol_{\mathcal{A}}(C) = (k - 1)vol_{\mathcal{M}_\delta}(C)$, and the same equality holds for $\bar{C}$. Consequently, $\phi_{\mathcal{M}_\delta}(C) \le \phi_{\mathcal{A}}(C) \le \beta_k \phi_{\mathcal{M}_\delta}(C), \beta_k = \frac{\lfloor k^2/4 \rfloor}{k-1}$. Let $C_{\mathcal{A}}$ be the result of the standard spectral sweep on $\mathcal{A}$. By the classical Cheeger inequality [1], $\phi_{\mathcal{M}_\delta}(C_{\mathcal{A}}) \le \phi_{\mathcal{A}}(C_{\mathcal{A}}) \le 2\sqrt{\phi^*_{\mathcal{A}}} \le 2\sqrt{\beta_k \phi^*_{\mathcal{M}_\delta}}$. Thus, the standard pairwise projection also admits an $O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$ guarantee for any fixed motif size.

A.3 Missing Proofs

Proof of Theorem 2.1. We first prove the case of escaping from C to $\bar{C}$. By Definition 7, if the walk starts from a node $u \in C$, then the probability of moving to $\bar{C}$ in one step is $\Pr(w_1 \in \bar{C} \mid w_0 = u) = \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i \,\&\, u \in S)}{d_{\mathcal{M}_\delta}(u)}$. Therefore, we have $\Pr(w_1 \in \bar{C} \mid w_0 \in C) = \sum_{u \in C} \Pr(w_1 \in \bar{C} \mid w_0 = u) \Pr(w_0 = u \mid w_0 \in C) = \sum_{u \in C} \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i \,\&\, u \in S)}{d_{\mathcal{M}_\delta}(u)} \frac{d_{\mathcal{M}_\delta}(u)}{vol_{\mathcal{M}_\delta}(C)} = \frac{\sum_{u \in C} \sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i \,\&\, u \in S)}{vol_{\mathcal{M}_\delta}(C)} = \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i) \sum_{u \in C} I(u \in S)}{vol_{\mathcal{M}_\delta}(C)} = \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i) \cdot i}{vol_{\mathcal{M}_\delta}(C)} = \frac{\sum_{i=1}^{k-1} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i)}{vol_{\mathcal{M}_\delta}(C)} = \frac{\sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(S \cap C \neq \emptyset \,\&\, S \cap \bar{C} \neq \emptyset)}{vol_{\mathcal{M}_\delta}(C)} = \frac{cut_{\mathcal{M}_\delta}(C)}{vol_{\mathcal{M}_\delta}(C)}$. Analogously, for the walk starting from $\bar{C}$, we have $\Pr(w_1 \in C \mid w_0 \in \bar{C}) = \frac{cut_{\mathcal{M}_\delta}(C)}{vol_{\mathcal{M}_\delta}(\bar{C})}$. Finally, according to Definition 6, we have $\phi_{\mathcal{M}_\delta}(C) = \frac{cut_{\mathcal{M}_\delta}(C)}{\min\{vol_{\mathcal{M}_\delta}(C), vol_{\mathcal{M}_\delta}(\bar{C})\}} = \max\left\{\frac{cut_{\mathcal{M}_\delta}(C)}{vol_{\mathcal{M}_\delta}(C)}, \frac{cut_{\mathcal{M}_\delta}(C)}{vol_{\mathcal{M}_\delta}(\bar{C})}\right\}$. Combining the above equalities completes the proof.

Proof of Theorem 2.2. The NP-hardness of minimum static motif conductance follows from the hardness of classical minimum graph conductance [6]. Importantly, this hardness concerns the minimization of conductance over candidate vertex subsets, rather than the computational complexity of motif-instance matching. Specifically, to prove that *MgcCT* is NP-hard, we reduce the minimum motif conductance problem to our *MgcCT* problem. Specifically, (1) *Instance Construction*: Let $G(V, E)$ be a de-temporal (i.e., static) graph and M be a de-temporal (i.e., static) motif. The minimum motif conductance problem is defined as finding a cluster $C \subseteq V$ that minimizes $\phi_M(C) = \frac{cut_M(C)}{min(vol_M(C), vol_M(\overline{C}))}$, where $cut_M(C)$ denotes the number of M-instances crossing the boundary between C and $\overline{C}$, and $vol_M(C)$ denotes the total number of M-instances involving nodes in C. (2) *Reduction to MgcCT*: Construct a temporal graph $\mathcal{G}(V, \mathcal{E})$ from $G(V, E)$ by assigning

all edges the same timestamp t_0. Define a δ-temporal motif $\mathcal{M}_\delta$ as the temporal extension of M with $\delta = 0$, ensuring that $\mathcal{M}_\delta$-instances in $\mathcal{G}$ are exactly equivalent to M-instances in G. (3) *Equivalence Verification*: For any cluster $C \subseteq V$, the temporal motif conductance of C in $\mathcal{G}$ (Definition 6) satisfies: $\phi_{\mathcal{M}_\delta}(C) = \frac{cut_{\mathcal{M}_\delta}(C)}{min(vol_{\mathcal{M}_\delta}(C), vol_{\mathcal{M}_\delta}(\bar{C}))} = \phi_M(C)$ Thus, finding the cluster C^* that minimizes $\phi_{\mathcal{M}_\delta}(C)$ (the *MgcCT* problem) is equivalent to finding the cluster C^* that minimizes $\phi_M(C)$ (the minimum motif conductance problem). (4) *Conclusion*: Since the minimum motif conductance problem is NP-hard [6], and the above reduction can be completed in polynomial time, the *MgcCT* problem is also NP-hard.

Proof of Lemma 4.1. For any temporal motif instance $\mathcal{G}_S \in \mathcal{M}_\delta$, we let $H_S = (S, E_S^{\mathrm{dt}})$ be its directed de-temporal graph. Define $b_C(\mathcal{G}_S) = \left|\left\{(x,y) \in E_S^{\mathrm{dt}} \mid x \in S \cap C,\ y \in S \cap \bar{C} \text{ or } x \in S \cap \bar{C},\ y \in S \cap C\right\}\right|$. Namely, $b_C(\mathcal{G}_S)$ is the number of directed de-temporal edges of $\mathcal{G}_S$ whose endpoints are separated by the cut $(C, \bar{C})$, regardless of edge directions. By Definition 9 and $W = W_{ef} + W_{ef}^\top$, we have $\sum_{u\in C,\, v\notin C} W_{uv} = \sum_{u\in C,\, v\notin C}\left((W_{ef})_{uv} + (W_{ef})_{vu}\right) = \sum_{u\in C,\, v\notin C}\sum_{\mathcal{G}_S\in\mathcal{M}_\delta}\left(\mathbb{I}((u,v)\in E_S^{\mathrm{dt}}) + \mathbb{I}((v,u)\in E_S^{\mathrm{dt}})\right) = \sum_{\mathcal{G}_S\in\mathcal{M}_\delta} b_C(\mathcal{G}_S)$. If $\mathcal{G}_S$ is not cut by C, namely $S\cap C = \emptyset$ or $S\cap\bar{C} = \emptyset$, then $b_C(\mathcal{G}_S) = 0$. Otherwise, the cut $(S\cap C, S\cap\bar{C})$ induces a nonempty proper partition of the motif vertices under the isomorphism between $\mathcal{M}_\delta$ and $\mathcal{G}_S$. Therefore, by the definitions of $\underline{c}_\delta$ and $\bar{c}_\delta$, we have $\underline{c}_\delta \le b_C(\mathcal{G}_S) \le \bar{c}_\delta$. Consequently, $\underline{c}_\delta \sum_{\mathcal{G}_S\in\mathcal{M}_\delta}\mathbb{I}(S\cap C\neq\emptyset,\ S\cap\bar{C}\neq\emptyset) \le \sum_{\mathcal{G}_S\in\mathcal{M}_\delta} b_C(\mathcal{G}_S) \le \bar{c}_\delta\sum_{\mathcal{G}_S\in\mathcal{M}_\delta}\mathbb{I}(S\cap C\neq\emptyset,\ S\cap\bar{C}\neq\emptyset)$. By Definition 3, the summation on both sides is exactly $\mathrm{cut}_{\mathcal{M}_\delta}(C)$. Thus, $\underline{c}_\delta\,\mathrm{cut}_{\mathcal{M}_\delta}(C) \le \sum_{u\in C,\, v\notin C} W_{uv} \le \bar{c}_\delta\,\mathrm{cut}_{\mathcal{M}_\delta}(C)$, which completes the proof.

Proof of Lemma 4.2. For a temporal motif instance $\mathcal{G}_S \in \mathcal{M}_\delta$ and a vertex $u \in V$, define $a_u(\mathcal{G}_S) = \left|\left\{(u,v)\in E_S^{\mathrm{dt}} \mid v\in S\right\}\right| + \left|\left\{(v,u)\in E_S^{\mathrm{dt}} \mid v\in S\right\}\right|$. That is, $a_u(\mathcal{G}_S)$ is the number of directed de-temporal edges of $\mathcal{G}_S$ incident to u, regardless of edge directions. By Definition 9 and $W = W_{ef} + W_{ef}^\top$, for any vertex $u \in V$, we have $\sum_{v\in V} W_{uv} = \sum_{v\in V}\left((W_{ef})_{uv} + (W_{ef})_{vu}\right) = \sum_{v\in V}\sum_{\mathcal{G}_S\in\mathcal{M}_\delta}\left(\mathbb{I}((u,v)\in E_S^{\mathrm{dt}}) + \mathbb{I}((v,u)\in E_S^{\mathrm{dt}})\right) = \sum_{\mathcal{G}_S\in\mathcal{M}_\delta} a_u(\mathcal{G}_S)$. If $u\notin S$, then $a_u(\mathcal{G}_S) = 0$. If $u\in S$, then u corresponds to some motif vertex $x\in V_\delta$ under the isomorphism between $\mathcal{M}_\delta$ and $\mathcal{G}_S$. Hence, $a_u(\mathcal{G}_S) = d_{H_\delta}(x)$. By the definitions of $\underline{d}_\delta$ and $\bar{d}_\delta$, we have $\underline{d}_\delta \le a_u(\mathcal{G}_S) \le \bar{d}_\delta$ whenever $u\in S$. Equivalently, $\underline{d}_\delta\,\mathbb{I}(u\in S) \le a_u(\mathcal{G}_S) \le \bar{d}_\delta\,\mathbb{I}(u\in S)$. Summing over all $\mathcal{G}_S\in\mathcal{M}_\delta$ and all $u\in C$, we obtain $\underline{d}_\delta\,\mathrm{vol}_{\mathcal{M}_\delta}(C) = \underline{d}_\delta\sum_{u\in C}\sum_{\mathcal{G}_S\in\mathcal{M}_\delta}\mathbb{I}(u\in S) \le \sum_{u\in C}\sum_{\mathcal{G}_S\in\mathcal{M}_\delta} a_u(\mathcal{G}_S) = \sum_{u\in C}\sum_{v\in V} W_{uv} \le \bar{d}_\delta\sum_{u\in C}\sum_{\mathcal{G}_S\in\mathcal{M}_\delta}\mathbb{I}(u\in S) = \bar{d}_\delta\,\mathrm{vol}_{\mathcal{M}_\delta}(C)$. This completes the proof.

Proof of Theorem 4.1. Initially, Algorithm 1 requires $O(C(\mathcal{M}_\delta))$ time to construct the edge-focused weighted matrix W_{ef} (Lines 2-4). The symmetrization step $W \leftarrow W_{ef} + W_{ef}^\top$ takes $O(\bar{m})$ time (Line 5) since W_{ef} is supported on the edges of the de-temporal graph G. Next, constructing $\mathcal{D}$ costs $O(\bar{m})$ time (Lines 6-7), and forming the Laplacian matrix $\mathcal{L}$ costs $O(\bar{m})$ time (Line 8). Subsequently, Algorithm 1 takes $T_{eig}^{(2)}(W)$ to compute the eigenvector associated with the second smallest eigenvalue (Line 9). Finally, the algorithm takes $O(n\log n)$ time to sort the indices (Line 10) and $O(\bar{m})$ time to calculate the conductance of C_i on the weighted graph W (Line 11). Consequently, the overall time complexity of Algorithm 1 is $O\left(C(\mathcal{M}_\delta) + T_{eig}^{(2)}(W) + \bar{m} + n\log n\right)$.

For space complexity, Algorithm 1 needs an additional $O(\bar{m})$ space to store W_{ef}, W, $\mathcal{D}$, and $\mathcal{L}$. Additionally, it requires $O(m)$ space to store the input temporal graph $\mathcal{G}$. Hence, the total space complexity of Algorithm 1 is $O(m)$ due to $m > \bar{m}$.

Proof of Theorem 4.2. For any cluster $C\subseteq V$, the edge conductance on W is $\phi_W(C) = \frac{\sum_{u\in C,\, v\notin C} W_{uv}}{\min\{\sum_{u\in C}\sum_{v\in V} W_{uv}, \sum_{u\in\bar{C}}\sum_{v\in V} W_{uv}\}}$ [21]. By Lemma 4.1, we have $\underline{c}_\delta\,\mathrm{cut}_{\mathcal{M}_\delta}(C) \le \sum_{u\in C,\, v\notin C} W_{uv} \le \bar{c}_\delta\,\mathrm{cut}_{\mathcal{M}_\delta}(C)$. By Lemma 4.2, we have $\underline{d}_\delta\,\mathrm{vol}_{\mathcal{M}_\delta}(C) \le \sum_{u\in C}\sum_{v\in V} W_{uv} \le \bar{d}_\delta\,\mathrm{vol}_{\mathcal{M}_\delta}(C)$, and similarly, $\underline{d}_\delta\,\mathrm{vol}_{\mathcal{M}_\delta}(\bar{C}) \le \sum_{u\in\bar{C}}\sum_{v\in V} W_{uv} \le \bar{d}_\delta\,\mathrm{vol}_{\mathcal{M}_\delta}(\bar{C})$. Combining these inequalities, we have

$$\frac{\underline{c}_\delta}{\bar{d}_\delta}\phi_{\mathcal{M}_\delta}(C) \le \phi_W(C) \le \frac{\bar{c}_\delta}{\underline{d}_\delta}\phi_{\mathcal{M}_\delta}(C). \tag{2}$$

Let $\phi_W^* = \min_{C\subseteq V}\phi_W(C)$. By the classical Cheeger inequality for undirected weighted graphs [1], we have

$$\frac{\lambda_2}{2} \le \phi_W^* \le \sqrt{2\lambda_2}. \tag{3}$$

Let $C^*_{\mathcal{M}_\delta}$ be an optimal solution for temporal motif conductance. By Eq.2, $\phi_W^* \le \phi_W(C^*_{\mathcal{M}_\delta}) \le \frac{\bar{c}_\delta}{\underline{d}_\delta}\phi_{\mathcal{M}_\delta}(C^*_{\mathcal{M}_\delta}) = \frac{\bar{c}_\delta}{\underline{d}_\delta}\phi^*_{\mathcal{M}_\delta}$. Together with $\lambda_2/2 \le \phi_W^*$ from Eq.3, we obtain $\frac{\underline{d}_\delta}{2\bar{c}_\delta}\lambda_2 \le \phi^*_{\mathcal{M}_\delta}$. Next, let C_W^* be an optimal solution for ϕ_W. By Eq.2, $\phi_{\mathcal{M}_\delta}(C_W^*) \le \frac{\bar{d}_\delta}{\underline{c}_\delta}\phi_W(C_W^*) = \frac{\bar{d}_\delta}{\underline{c}_\delta}\phi_W^*$. Since $\phi^*_{\mathcal{M}_\delta} \le \phi_{\mathcal{M}_\delta}(C_W^*)$ and $\phi_W^* \le \sqrt{2\lambda_2}$ by Eq.3, we have $\phi^*_{\mathcal{M}_\delta} \le \frac{\bar{d}_\delta}{\underline{c}_\delta}\sqrt{2\lambda_2}$. Combining the two bounds completes the proof.

Proof of Theorem 4.3.

Let C be the set returned by Algorithm 1, which minimizes ϕ_W over the nonempty proper prefix sets induced by its direct eigenvector ordering. As in the proof sketch, the transfer argument starts from the graph-level sweep bound stated in Theorem 2 of [43] and the discussion following Theorem 1 in Section 2 of [72]:

$$\phi_W(C) \le \sqrt{2\lambda_2}. \tag{4}$$

By Eq.2 in the proof of Theorem 4.2, for any cluster C, we have

$$\phi_{\mathcal{M}_\delta}(C) \le \frac{\bar{d}_\delta}{\underline{c}_\delta}\phi_W(C). \tag{5}$$

Combining Eq.4 and Eq.5, we obtain

$$\phi_{\mathcal{M}_\delta}(C) \le \frac{\bar{d}_\delta}{\underline{c}_\delta}\sqrt{2\lambda_2}. \tag{6}$$

By the Cheeger inequality [1], $\lambda_2/2 \le \phi_W^*$, which implies $\lambda_2 \le 2\phi_W^*$. Moreover, applying Eq.2 to the optimal temporal-motif-conductance cluster $C^*_{\mathcal{M}_\delta}$ gives

$$\phi_W^* \le \phi_W(C^*_{\mathcal{M}_\delta}) \le \frac{\bar{c}_\delta}{\underline{d}_\delta}\phi^*_{\mathcal{M}_\delta}. \tag{7}$$

Table 5: Motif-dependent computational costs and approximation constants. m is the number of temporal edges, and b_δ is the maximum number of temporal edges in any duration-δ window.

Motif	M_1	M_2	M_3	M_4	M_5	M_6
r_i	3	3	3	4	4	4
$Time_{\text{Mackey}}(M_i)$	$O(mb_\delta^2)$	$O(mb_\delta^2)$	$O(mb_\delta^2)$	$O(mb_\delta^3)$	$O(mb_\delta^3)$	$O(mb_\delta^3)$
$\underline{c}_\delta$	2	2	1	2	1	1
$\overline{c}_\delta$	2	2	3	4	4	4
$\underline{d}_\delta$	2	2	1	2	1	1
$\overline{d}_\delta$	2	2	2	2	3	2
K_W	2.00	2.00	6.93	2.83	12.00	8.00

Table 6: A comprehensive comparison of the SOTA methods. Higher-order structures denote whether the objective function considers the informative motifs. Directed denotes whether the edges are directed or not. Note that *THDS* [64] only considers the internal higher-order cohesiveness and overlooks the critical external sparsity, which is an important factor in measuring the quality of a cluster [25, 31, 36, 41], resulting in unsatisfactory quality as stated in our empirical results. Our model *MgcCT* avoids this issue (details in Definition 6).

Methods	Continuous-time	Higher-order	Directed	Clustering Function
TMO [55]	×	×	×	Modularity-based
TSC [66]	×	×	×	Conductance-based
TDS[58]	✓	×	×	Density-based
HMO [4, 29]	×	✓	✓	Modularity-based
HSC [6, 30, 72]	×	✓	✓	Conductance-based
HDS [13, 26, 68]	×	✓	✓	Density-based
THDS[64]	✓	✓	✓	Density-based
MgcCT (our)	✓	✓	✓	Conductance-based

Substituting Eq.7 into Eq.6, we have $\phi_{\mathcal{M}_\delta}(C) \le \frac{\overline{d}_\delta}{\underline{c}_\delta}\sqrt{2\lambda_2} \le \frac{\overline{d}_\delta}{\underline{c}_\delta}\sqrt{4\phi^*_W} \le 2\frac{\overline{d}_\delta}{\underline{c}_\delta}\sqrt{\frac{\overline{c}_\delta}{\underline{d}_\delta}\phi^*_{\mathcal{M}_\delta}}$. Since $\underline{c}_\delta$, $\overline{c}_\delta$, $\underline{d}_\delta$, and $\overline{d}_\delta$ are constants determined only by the query temporal motif, we have $\phi_{\mathcal{M}_\delta}(C) = O(\sqrt{\phi^*_{\mathcal{M}_\delta}})$ for fixed-size temporal motifs.

Proof of Theorem 4.4. The time cost of Algorithm 2 is mainly consumed in calculating the edge-focused weighted degree of nodes (Line 7) and performing push operations on nodes (Line 8). Let u_i be the node to be pushed in the ith push operation and n_i be the number of nodes that receive propagation during the i-th push operation is performed. When the ith push operation is performed (i.e., $r(u) \ge \epsilon \cdot EFD(u)$ in Line 20), the residue $r(u_i) \ge \epsilon n_i$ due to $EFD(u_i) \ge n_i$, so $\|r\|_1$ decreased by at least $\alpha\epsilon n_i$ (Lines 21-24). Since $\|r\|_1 = 1$ initially (Line 1) and $\|r\|_1$ is at most 1 throughout the entire push process because r always decreases (Lines 22 and 24), we have $\alpha\epsilon \sum_{i=1} n_i \le 1$ and $\sum_{i=1} n_i \le \frac{1}{\alpha\epsilon}$. Therefore, the time complexity of all push operations in Algorithm 2 is $O(\frac{1}{\alpha\epsilon})$. On the other hand, we only calculate the edge-focused weighted degree $EFD(u)$ for vertex u in the queue Q instead of all nodes in advance. Thus, by Line 23, a total of $O(\frac{1}{\alpha\epsilon})$ vertices have ever been in Q. As a result, the time complexity of all calculations of the edge-focused weighted degree in Algorithm 2 is $O(\frac{\Delta_{max}}{\alpha\epsilon})$. The sorting and sweep-cut costs over the nonzero entries of π are local operations and are dominated by the on-demand edge-focused weight computations. So, the overall time complexity of Algorithm 2 is $O(\frac{\Delta_{max}}{\alpha\epsilon})$. Regarding space complexity, through the above proof process, we find that Algorithm 2 visits at most $O(\frac{1}{\alpha\epsilon})$ nodes, so it requires $O(\frac{1}{\alpha\epsilon})$ space to maintain r, π, Q, and D. Hence, the space complexity of Algorithm 2 is $O(\frac{1}{\alpha\epsilon})$.

A.4 Motif-Dependent Complexity and Approximation Guarantees

Let r_i denote the number of temporal edges in motif M_i, and let b_δ be an upper bound on the number of temporal edges within any duration-δ window. For the chronological matching backend of Mackey et al. [52], a conservative enumeration-time bound is $O(mb_\delta^{r_i-1})$. These bounds characterize the enumeration-based implementation, rather than the potential cost of a specialized edge-conditioned counting algorithm. Moreover, the approximation coefficient in Theorem 4.3 is $K_W = 2\frac{\overline{d}_\delta}{\underline{c}_\delta}\sqrt{\frac{\overline{c}_\delta}{\underline{d}_\delta}}$. Larger values of K_W correspond to weaker worst-case approximation bounds. They do not necessarily imply worse observed clustering quality. Table 5 summarizes the motif-dependent enumeration costs and approximation constants for all six query motifs. The enumeration cost increases with the number of temporal edges in the query motif. In contrast, the approximation coefficient K_W depends on the motif-specific cut and volume comparison constants.

A.5 Advantages of *MgcCT*

(1) According to Definition 6, we know that C^* is the sparsest balanced cluster, in which the sparsest signifies to minimize the δ-instances that cross C^* and $V \setminus C^*$, and the balanced indicates the δ-instances that are in the cluster. Thus, minimizing temporal motif conductance can find a cluster that preserves seamlessly higher-order temporal connectivity. (2) It provides a new clustering metric based on higher-order temporal motifs rather than simple temporal edges, allowing users to flexibly focus the clusters that highlight specific temporal motifs relevant to a given type of temporal network. For instance, temporal cycles are crucial features of financial markets, and emphasizing these clusters with dense temporal cycles can provide strong interpretability and improve the clustering quality. (3) The capability to handle directed temporal graphs for clustering is a largely advantage of our solutions, while existing methods are limited to undirected temporal graphs [47, 55, 58, 66, 81, 83]. For example, by the definition of δ-temporal motif (Definition 1), there are eight possible triangles on directed temporal graphs, but only one triangle exists on undirected temporal graphs, significantly missing this rich information. (4) It also admits an exact cluster-conditioned one-step escape-probability interpretation (Theorem 2.1), offering a complementary probabilistic perspective on why a cluster with small temporal motif conductance tends to preserve temporal motif instances internally while cutting few such instances across the boundary. Table 6 reports the comprehensive comparison of the state-of-the-art (SOTA) graph clustering methods.

A.6 Practical Interpretations of Temporal Motifs

The six temporal motifs in Fig. 4 capture different higher-order interaction patterns, which may reflect distinct behaviors in real-world temporal networks. The following gives some practical interpretations for them.

M_1**: Feed-forward interactions.** M_1 represents a directed triangular pattern in which two nodes interact directly, followed by an indirect two-step interaction through an intermediate node. Such patterns can reflect information reinforcement or indirect confirmation in social and communication networks. For example, a user may first contact another user directly and then convey related information through a mutual contact. Clusters enriched with M_1 may therefore capture groups exhibiting repeated or reinforced communication relationships.

M_2**: Cyclic interactions.** M_2 represents a three-node directed temporal cycle, in which interactions eventually return to the starting node. Such patterns may indicate feedback or reciprocal information circulation in social and collaboration networks. In financial or trust networks, they can also provide structural signals for examining circular interactions among multiple participants.

M_3**: Sequential propagation.** M_3 represents a temporal path involving three consecutive interactions among four distinct nodes. This motif is useful for characterizing sequential information or influence propagation, such as rumor spreading in social networks or multi-step communication in organizational systems. Clusters enriched with M_3 may reveal groups containing closely connected propagation chains or potential spreading cascades.

M_4**: Higher-order temporal cycles.** M_4 extends cyclic interactions to four distinct nodes, forming a temporally ordered directed cycle. Compared with M_2, it captures longer chains of interactions that eventually return to the starting node. Such patterns may reflect multi-party feedback, circular communication, or recurrent interactions in financial and collaboration networks. They can also provide candidate structures for further investigation of unusual circular activities.

M_5**: Converging and forwarding interactions.** M_5 represents a temporal structure in which two different sources contact a common node, followed by a sequence of outgoing interactions. For example, two individuals may report information to a coordinator, who subsequently forwards it through other participants. Such patterns may characterize information aggregation and subsequent propagation in organizational, collaboration, or communication networks. In cybersecurity applications, similar structures may also motivate the investigation of coordinated requests followed by downstream activities.

M_6**: Interconnected convergence patterns.** M_6 captures two temporally ordered many-to-one interaction patterns connected through a shared sending node. Specifically, two sources first contact one recipient, while the shared source subsequently contacts another recipient that also receives an interaction from a different source. This structure may represent related reporting, shared participation, or coordinated communication across different groups. For example, in organizational networks, a participant may contribute information to multiple teams that also receive inputs from other members. Such patterns can help characterize interconnected coordination structures beyond simple temporal paths or cycles.

Overall, these motifs capture complementary temporal interaction patterns, including reinforcement, feedback, sequential propagation, cyclic interactions, information aggregation, and interconnected convergence. Their practical interpretations depend on the application and the semantics of temporal edges. By considering different query motifs, MgcCT can identify clusters that preserve different types of higher-order temporal relationships.

A.7 Discussion of Motif Topology and Duration

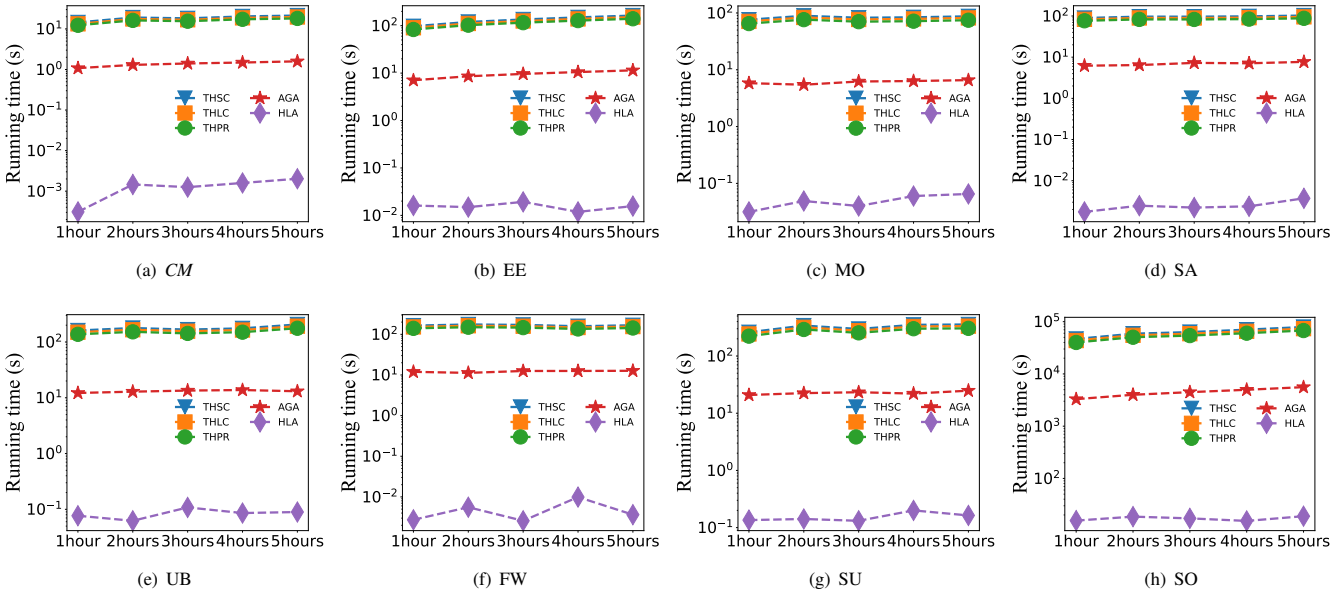
The six evaluated query motifs cover different temporal-edge counts, vertex counts, and topological patterns. Specifically, M_1–M_3 contain three temporal edges, whereas M_4–M_6 contain four temporal edges. Their different topologies affect both the numbers of valid temporal-motif instances and the motif-dependent approximation constants. The runtime results for all six motifs illustrate that computational cost depends on motif size and topology; it is not determined by the number of temporal edges alone. The duration threshold δ also affects the set of valid temporal-motif instances. For a fixed graph and motif, increasing δ cannot remove a previously valid instance, although the resulting TMC values and returned clusters need not change monotonically. The experiments use $\delta = 1$ hour by default, with additional duration settings considered in the supplementary evaluation. We therefore limit our conclusions to the evaluated motif and duration settings rather than claiming uniform insensitivity.

A.8 Additional Experiments

The following additional experiments are provided to complement the main empirical results. In particular, they examine whether the proposed methods remain effective and efficient when varying the temporal duration δ and the query temporal motif. These experiments are intended to verify that the empirical advantage of *AGA* and *HLA* is not limited to the default setting.

Exp-8: Running time of different temporal motif conductance algorithms with varying δ. Figure 11 shows the running time of *THSC*, *THLC*, *THPR*, *AGA*, and *HLA* with varying the temporal duration δ. As can be seen, we have (1) the consistent speed advantage of *HLA* stems from its purely local design. Rather than precomputing the weight $(W_{ef})uv$ for all edges (u, v) in advance like other approaches, it calculates $(Wef)_{uv}$ on the fly when edge (u, v) is accessed. As a result, the runtime of *HLA* mainly depends on the locally explored region and the locally computed edge-focused weights, rather than on the overall graph scale. (Table 1). (2) Across all tested datasets and δ values, *AGA* consistently ranks as the second best performing method. Its efficiency is largely due to an innovative edge-centric weighted graph projection technique, which can be implemented effectively through the well-established temporal motif counting framework. In contrast, methods such as *THSC*, *THLC*, and *THPR* rely on exhaustively enumerating every temporal motif instance, leading to substantially longer execution times (Table 1). (3) All algorithms exhibit relatively stable performance as δ increases. This is because their time complexity mainly comes from the enumeration or counting of the temporal motif instance, but when δ increases within a relatively small range, it will not significantly affect their efficiency. In a nutshell, these results give preliminary evidence that the proposed solutions are highly efficient in practice.

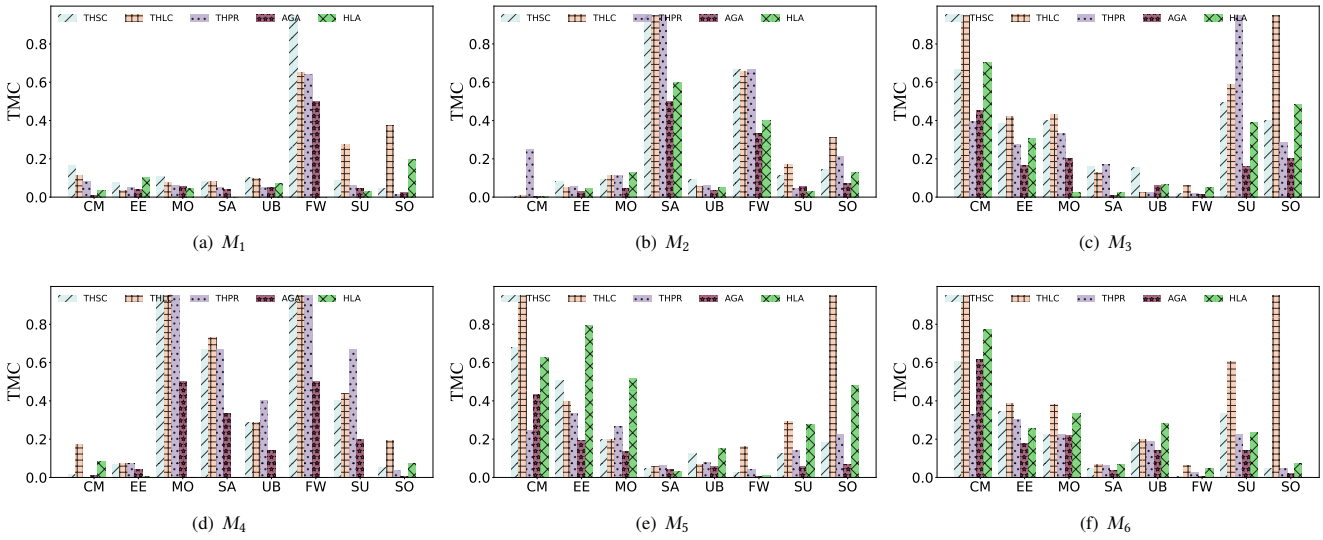
Exp-9: Effectiveness of different temporal motif conductance algorithms with varying parameters. Based on the experimental results presented in Figures 12 and 13, the following key observations can be drawn: (1) The proposed algorithms, *AGA* and *HLA*, demonstrate consistent superiority over existing baseline methods across a wide range of experimental configurations, including diverse temporal motif structures and varying temporal duration δ.



(a) *CM* (b) EE (c) MO (d) SA

(e) UB (f) FW (g) SU (h) SO

Figure 11: Running time of different temporal motif conductance algorithms with varying δ.

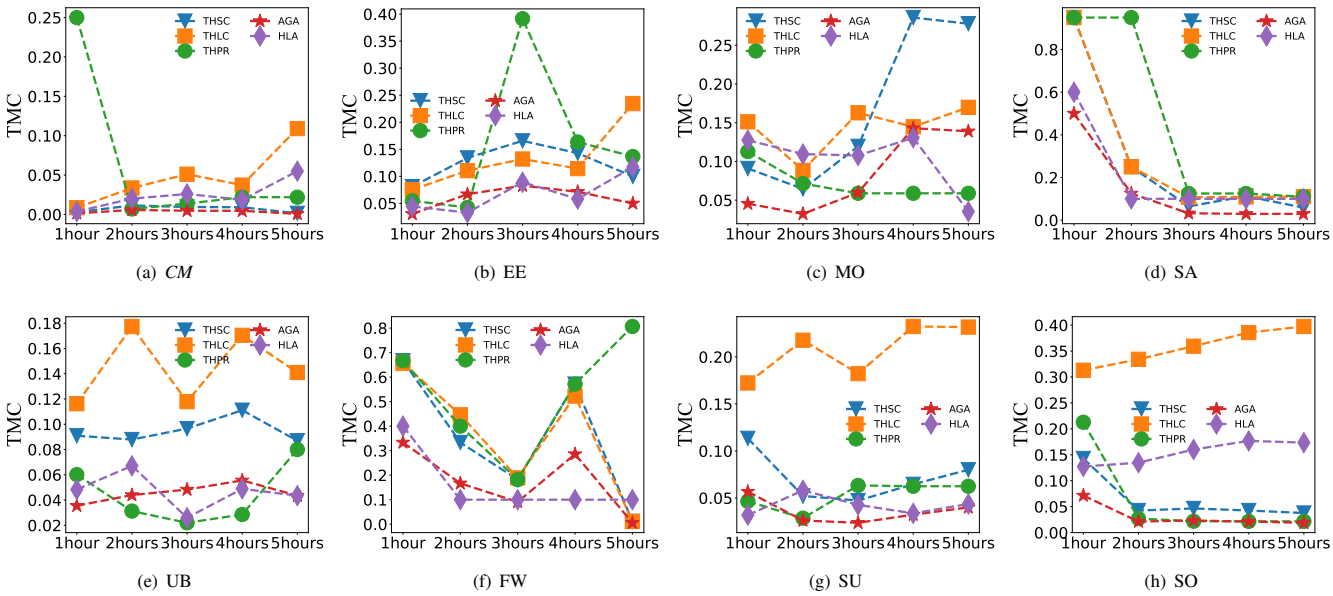


(a) M_1 (b) M_2 (c) M_3

(d) M_4 (e) M_5 (f) M_6

Figure 12: Effectiveness of different temporal motif conductance with varying temporal motif.

Notably, despite the absence of theoretical guarantees, *HLA* achieves performance levels comparable to those of *AGA* in most scenarios. This suggests that the HLA maintains competitive TMC values, highlighting its value in real-world applications where theoretical optimality may be secondary to empirical performance. (2) On average, the performance of our proposed *AGA* and *HLA* algorithms exhibit relative stability. This stability is evident across different datasets, temporal motifs, and temporal durations δ. In contrast, the performance of other baseline methods displays considerable variability, which is highly dependent on the specific characteristics of the dataset, temporal motif, and temporal duration δ. These findings provide further empirical evidence supporting the notion that our algorithms are adept at identifying clusters of higher quality compared to existing baseline methods when applied to real-world graph structures. This empirical validation aligns closely with the theoretical analysis presented in Table 1, reinforcing the theoretical underpinnings of our proposed methodologies.



(a) *CM* (b) EE (c) MO (d) SA

(e) UB (f) FW (g) SU (h) SO

Figure 13: Effectiveness of different temporal motif conductance with varying δ.

Exp-10: Ground-truth cluster recovery. To provide an objective-independent evaluation, we add a controlled synthetic experiment in which the target cluster is known *a priori*. We then evaluate whether each method can recover this externally specified cluster using F1 and normalized mutual information (NMI), neither of which is optimized by our algorithms. Specifically, for each synthetic instance, we generate a directed continuous-time temporal graph $\mathcal{G} = (V, \mathcal{E})$ with $|V| = 10,000$ vertices and randomly select a subset $C^* \subset V$ with $|C^*| = 500$ as the planted ground-truth cluster. We first generate $100,000$ background directed temporal edges by uniformly sampling their endpoints from the whole vertex set, subject to the no-self-loop constraint, and their timestamps from a 30-day observation interval. Before motif injection, vertices inside and outside C^* follow exactly the same background dyadic edge-generation process. We then introduce additional valid temporal motif instances according to the planted-cluster and noise generation procedure described below. To avoid drawing conclusions from a single motif topology, we separately construct synthetic graphs for three representative query motifs, M_2, M_4, and M_5, which cover different motif sizes and topological patterns considered in our real-data experiments. For each query motif, we generate 20,000 motif-generation events, each of which inserts one valid instance following the corresponding edge directions and temporal ordering. We use $\delta = 1$ hour, consistent with the default setting in our real-data experiments.

To vary recovery difficulty, we use $\eta \in \{0.1, 0.3, 0.5\}$. For each injected motif event, with probability $1 - \eta$ the instance lies completely inside C^*; with probability $\eta/2$ one motif vertex is sampled outside C^*, producing a boundary-crossing instance; with the remaining probability $\eta/2$ the instance lies completely outside C^* and acts as a distractor. Thus, increasing η decreases the fraction of internally planted motif events while increasing the number of boundary-crossing and outside-cluster distractor events. We generate five independent graphs for each motif/noise combination, yielding $3 \times 3 \times 5 = 45$ synthetic graphs in total. Each returned cluster is evaluated against the planted cluster using F1 and NMI. Unlike the default source selection in Sec. 5.1, for this synthetic experiment, we randomly sample 50 source vertices from the planted ground-truth cluster C^* for THLC and HLA, using the same seeds for both methods. This setting evaluates the ability of seed-dependent methods to recover a target cluster given one of its members. The results are averaged over 15 independently generated synthetic graphs at each noise level, with F1 and NMI first averaged over the 50 queries for seed-dependent methods.

As shown in the Table 7, we have the following observations: First, the recovery performance of all methods decreases monotonically as the noise ratio increases. This pattern is consistent with the benchmark construction, in which increasing η introduces more boundary-crossing and outside-cluster motif instances, weakening the planted motif signal. Second, the temporal-motif-aware methods achieve stronger planted-cluster recovery than most lower-order temporal and static higher-order baselines under the evaluated synthetic generation settings. Third, AGA achieves the highest F1 at all three noise levels and the highest NMI at $\eta = 0.1$ and $\eta = 0.5$. HLA achieves the highest NMI at $\eta = 0.3$ and shows competitive recovery performance at the lower noise levels. However, its F1 at $\eta = 0.5$ is lower than those of THSC and THLC. These results complement the TMC-based evaluation by measuring agreement with externally specified planted membership.

Exp-11: Source-node sensitivity. Both THLC and HLA require a source vertex, making their clustering quality potentially sensitive to source selection. Thus, we further evaluate THLC and HLA using 50 randomly selected source vertices on each real-world dataset. Specifically, for each dataset, 50 source vertices are randomly

Table 9: Locality Analysis of HLA.

Dataset	Visited nodes	Visited edges	Median $\Delta(u)$	P90 $\Delta(u)$	Max $\Delta(u)$
SU	624 (0.330%)	6380 (0.749%)	1.8 ms	8.7 ms	44 ms
SO	97 (0.0038%)	7400 (0.021%)	4.6 ms	22.5 ms	188 ms

Table 7: Ground-truth Cluster Recovery.

Method	$\eta = 0.1$		$\eta = 0.3$		$\eta = 0.5$	
	F1	NMI	F1	NMI	F1	NMI
TSC	0.423	0.175	0.348	0.116	0.232	0.046
TMO	0.377	0.140	0.301	0.085	0.201	0.032
TDS	0.500	0.242	0.388	0.146	0.263	0.063
HSC	0.508	0.248	0.397	0.152	0.254	0.058
HMO	0.377	0.138	0.260	0.061	0.181	0.024
HDS	0.446	0.194	0.336	0.108	0.229	0.045
THDS	0.609	0.350	0.468	0.211	0.347	0.115
THSC	0.741	0.510	0.607	0.347	0.462	0.205
THLC	0.759	0.535	0.618	0.359	0.459	0.203
THPR	0.664	0.412	0.496	0.235	0.381	0.139
AGA	**0.833**	**0.645**	**0.695**	0.462	**0.543**	**0.281**
HLA	0.813	0.615	0.670	**0.486**	0.412	0.251

Table 8: Source-node Sensitivity.

Method	Stat.	CM	EE	MO	SA	UB	FW	SU	SO
THLC	Mean	0.010	0.050	0.120	0.950	0.060	0.640	0.170	0.320
	Std.	0.003	0.012	0.027	0.018	0.014	0.080	0.041	0.067
HLA	Mean	0.010	0.040	0.120	0.600	0.040	0.400	0.030	0.120
	Std.	0.002	0.009	0.021	0.071	0.010	0.053	0.007	0.021

sampled from the entire vertex set, without imposing any additional selection criteria, such as minimum degree, motif participation, or structural importance. The same 50 source vertices are used for THLC and HLA on each dataset to ensure paired comparisons. For each method and dataset, we report the mean and standard deviation of TMC over the 50 evaluated sources.

Table 8 reports the results under the default motif M_2 and $\delta = 1$ hour. As can be seen, we have the following three observations. First, HLA achieves a lower or equal mean TMC than THLC on all eight datasets under the evaluated random source-selection protocol. The improvement is particularly pronounced on SU (0.030 versus 0.170) and SO (0.120 versus 0.320). Second, HLA exhibits a smaller standard deviation than THLC on seven of the eight datasets. For instance, on SU, the standard deviation decreases from 0.041 to 0.007, and on SO, from 0.067 to 0.021. These measurements indicate that HLA's improvement in average TMC is generally not accompanied by greater source-node variability in the evaluated samples. Third, SA is an exception. Although HLA reduces the mean TMC from 0.950 to 0.600, its standard deviation increases from 0.018 to 0.071. This demonstrates that improved average clustering quality does not necessarily imply lower sensitivity to source selection. One possible explanation is heterogeneous temporal-motif participation across source neighborhoods, although confirming this explanation would require further source-level analysis. Overall, the results provide direct empirical evidence of source-node variability under random source selection. We interpret these findings as sample-based observations rather than a guarantee of source-independent stability or robustness for all vertices.

Exp-12: Locality Analysis of HLA. To further quantify the empirical locality of the proposed HLA, we report the statistics of $\Delta(u)$ on the two largest datasets SU and SO under the default setting M_2, $\delta = 1$ hour, $\alpha = 0.2$, and $\epsilon = 1/m$, where $\Delta(u)$ denotes the time required to compute the edge-focused weighted degree of an activated vertex u. For each dataset, HLA is executed from the same 50 randomly selected source vertices used in the quality evaluation. The numbers of visited vertices and visited de-temporal edges are averaged over these 50 queries, while the $\Delta(u)$ statistics are computed over all activated vertices encountered in the corresponding runs. The visited-node percentage is computed relative to n, whereas the visited-edge percentage is computed relative to $\bar{m}$, the number of edges in the de-temporal graph on which HLA performs local diffusion. We summarize the empirical distribution of $\Delta(u)$ using its median, 90th percentile (P90), and maximum. P90 denotes the 90th percentile, i.e., 90% of the activated vertices have a $\Delta(u)$ no larger than the reported P90 value.

As shown in Table 9, we have the following observations. First, HLA visits only a small fraction of the graph under the evaluated queries. On SU, it visits on average 624 vertices and 6,380 de-temporal edges, corresponding to 0.330% of the vertices and 0.749% of the de-temporal edges. On SO, the corresponding fractions are only 0.0038% and 0.021%. Second, $\Delta(u)$ exhibits a noticeable right tail. The median/P90 values are 1.8/8.7 ms on SU and 4.6/22.5 ms on SO, whereas the maximum values are 44 ms and 188 ms, respectively. Thus, $\Delta_{\max}$ reflects relatively expensive local vertices and can be substantially larger than the cost encountered for a typical activated vertex.

Exp-13: Runtime and memory breakdown. In this experiment, we decompose the total runtime/memory to identify where the efficiency difference arises. Specifically, we distinguish three components: (i) motif discovery, (ii) weight/statistic construction, and (iii) partitioning. "Motif discovery" measures chronological matching for valid temporal-motif instances, excluding method-specific updates. "Weight/statistic construction" covers the pairwise motif-weighted graph for THSC and THLC, motif-resident statistics for THPR, and edge-focused weights for AGA and HLA. "Partitioning" includes spectral computation and conductance sweeps for THSC and AGA, PPR diffusion and sweep for THLC, iterative peeling for THPR, and local diffusion and output for HLA. As shown in Table 10, for runtime breakdown, we have the following observations. (1), THSC, THLC, THPR, and AGA have comparable motif-discovery costs because they use the same general-purpose Mackey matcher. Therefore, the observed efficiency advantage of AGA does not originate from a faster motif-discovery backend. (2), the main runtime differences arise from weight/statistic construction and subsequent partitioning. THSC and THLC construct the conventional pairwise motif-weighted graph, which may introduce weighted connections absent from the original de-temporal graph. In contrast, AGA constructs the sparse edge-focused representation W_{ef}, whose support is restricted to the original $\bar{m}$ directed de-temporal edges.

Table 10: Runtime and memory breakdown.

Runtime (s)				
Method	Discovery	Construction	Partitioning	Total
THSC	3420	47500	88590	139510
THLC	3480	44700	78401	126581
THPR	3510	41000	76738	121248
AGA	3450	2560	3144	9154
HLA	3.6	1.2	5.2	10.0
Memory (MB)				
Method	Discovery	Construction	Partitioning	Total
THSC	14700	8100	2828	25628
THLC	14700	4400	1126	20226
THPR	425	70	228	723
AGA	430	45	223	698
HLA	410	30.5	202.5	643

The construction and partitioning costs are substantially lower for AGA. THPR follows a different computational procedure, incurring costs for motif-resident statistics and iterative peeling. (3) HLA avoids global motif discovery and global construction of W_{ef}. Instead, it computes the required edge-focused weights on demand during local diffusion, avoiding global materialization of the motif-weighted graph. This substantially reduces the processing required for the evaluated local queries. This accounts for its lower observed runtime under the evaluated queries. Overall, the runtime breakdown shows that the observed speedups should not be attributed to a faster general-purpose motif-discovery algorithm. Under the current implementation, the efficiency gains are associated with differences in representation construction and subsequent clustering, particularly the sparse edge-focused representation used by AGA. HLA further reduces the processing required per query through on-demand weight computation and local diffusion.

For memory breakdown, we can know that motif discovery and weight graph construction account for 22,800 MB (89.0%) of the THSC total and 19,100 MB (94.4%) of the THLC total. Although THSC, THLC, THPR, and AGA share the same motif-discovery backend, their auxiliary storage requirements differ because they retain and aggregate intermediate information differently. THSC and THLC construct pairwise motif-weighted representations, whereas THPR maintains motif-related statistics and AGA accumulates edge-focused weights. HLA additionally avoids global weighted-graph materialization. The measured aggregate totals indicate that AGA uses 36.7× less additional memory than THSC, while HLA uses 31.5× less than THLC in the reported setting. The structural support bound $\mathrm{nnz}(W_{ef}) \le \bar{m}$ (and $\mathrm{nnz}(W_{ef} + W_{ef}^{\top}) \le 2\bar{m}$) explains the sparse representation, but does not by itself determine peak memory.

A.9 Additional Related Work

Temporal motifs have been widely recognized as fundamental higher-order structural units in temporal networks, with extensive research exploring their properties and applications [8, 23, 32, 34, 49, 59, 62, 63, 74]. Early foundational work by Kovanen et al. [32] introduced the first temporal motif definition that explicitly incorporates Δ-adjacency constraints between consecutive events (i.e., temporal edges), where two events are considered Δ-adjacent if their timestamp difference does not exceed Δ. Gurukar et al. [23] investigated communication motifs as recurring subgraphs with consistent information flow patterns, while Paranjape et al. [59] proposed the δ-temporal motif, which incorporates both temporal duration and ordering constraints (Definition 1). This work has inspired numerous exact algorithms for mining specific 3-edge δ-temporal motifs [19, 38, 59, 60, 77], as well as sampling-based approximation methods [49, 74]. However, these small motifs face limitations in real-world applications [50, 52, 70, 80]. For arbitrary δ-temporal motifs with flexible node and edge configurations, Mackey et al. [52] developed the SOTA BT algorithm, an exact backtracking approach based on isomorphism testing. This has subsequently motivated hardware-accelerated implementations to optimize performance [70, 80]. Many randomized algorithms have been proposed for estimating motif counts [49, 62, 63, 74].